

DATA MINING

Cours expliqué pas à pas

Tous les exemples résolus en détail • scripts Python • projet Streamlit

Enseignant Th. K. DAGBA

Année 2025–2026

Contenu Chap. 1, 2, 4, 5 + TP classification + Projet

Table des matières

Mode d’emploi de ce document	2
1 Introduction au Data Mining	3
1.1 Pourquoi le Data Mining ? (motivations)	3
1.2 Qu’est-ce que le Data Mining ?	3
1.2.1 Trois types de requêtes	3
1.3 Les tâches du Data Mining	4
1.4 Le processus CRISP-DM	4
1.5 Outils et sources de données	5
2 Prétraitement des données	6
2.1 Pourquoi prétraiter ?	6
2.2 Données manquantes	6
2.3 Données bruitées	6
2.4 Intégration & redondance	7
2.5 Transformation des données	7
2.6 Réduction des données	7
2.7 Exercices résolus	8
2.7.1 Exercice 1 — repérer les problèmes de qualité	8
2.7.2 Exercice 2 — normalisation & standardisation (dataset churn)	8
2.8 Script Python commenté	8
3 Classification	12
3.1 Méthode des K plus proches voisins (K-NN)	12
3.1.1 Exercice résolu — classer David ($K = 3$, euclidienne)	12
3.2 Arbres de décision	17
3.2.1 Critère 1 — Indice de Gini	17
3.2.2 Critère 2 — Gain d’information (entropie)	21
3.3 Mesures de performance	26
4 Segmentation (Clustering)	28
4.1 Mesurer la qualité d’un cluster	28
4.2 Clustering hiérarchique	28
4.2.1 Exercice résolu — lien simple	28
4.3 Clustering par partitionnement — K-means	31
4.3.1 Exercice résolu — $C = \{2, 3, 4, 10, 11, 12, 20, 25, 30\}$, $K = 2$, $\mu_1 = 2$, $\mu_2 = 4$	31
5 TP « CLASSIFICATION » — corrigé complet	35
5.1 Exercice 1 — Indice de Gini	35
5.2 Exercice 2 — Gain d’information	35
5.3 Vérification sous WEKA	35
6 Le projet de Data Mining — application Streamlit	37
6.1 Le sujet	37
6.2 Architecture de la solution	37
6.3 Les 3 tâches dans l’application	37
6.4 Mesures de performance & comparaison WEKA	38
6.5 Code — algorithme Apriori (maison)	38
6.6 Code — application Streamlit	40

Mode d'emploi de ce document

Ce document reprend **l'intégralité** du cours de Data Mining et l'explique **de la façon la plus simple possible**. Pour chaque chapitre tu trouveras :

- la **théorie** reformulée en mots simples, avec des encadrés colorés ;
- **tous les exemples du cours résolus à la main**, chaque calcul détaillé ;
- le **script Python équivalent** (commenté ligne à ligne) qui refait le calcul puis le **vérifie avec scikit-learn** ;
- des **figures** générées par ces scripts.

Astuce / À retenir

Les codes des encadrés « listing » sont les vrais fichiers du dossier `scripts/`. Tu peux les exécuter avec, par exemple :

```
uv run python scripts/knn_david.py
```

Et l'application du projet avec : `uv run streamlit run projet/app.py`

En résumé : les 4 idées clés

1. Le Data Mining transforme des *données* en *connaissances* pour *décider*. **2.** Quatre tâches : *classification*, *prédiction*, *association*, *segmentation*. **3.** *Supervisé* (classe connue) vs *non supervisé* (classe inconnue). **4.** 60% du travail = *préparer les données*.

Introduction au Data Mining

1.1 Pourquoi le Data Mining ? (motivations)

Exemple — le banquier

Un client demande un crédit. Le banquier veut savoir *s'il va rembourser*. Le client ne dira jamais « je ne rembourserai pas ». Mais le banquier peut **inférer** le comportement futur du client à partir de la **base de données des anciens clients**. **C'est exactement ça le Data Mining** : utiliser le passé (les données) pour prédire/ comprendre le futur.

On vit une **explosion des données** : bases de données, entrepôts, capteurs, web, e-commerce, hôpitaux, supermarchés... On produit des téraoctets chaque jour, mais **seule une infime partie est analysée**. D'où le besoin d'outils capables d'extraire la *connaissance* cachée dans ces montagnes de données.

Astuce / À retenir

Phrase à retenir du cours : « *Nous sommes inondés de données, et il nous manque les connaissances contenues dans ces données.* »

1.2 Qu'est-ce que le Data Mining ?

1.2.1 Trois types de requêtes

- **Opérationnelle** (base de données / SQL) : « *Combien de bouteilles de vin vendues au 1^{er} trimestre 2010 à la boutique EREVAN ?* »
- **Analytique** (entrepôt / OLAP) : « *ventes par département, par marque, par trimestre sur 5 ans* » — agrégation à différents niveaux.
- **Exploratoire** (Data Mining) : « *Peut-on **prédire** qu'un client va acheter du vin ? Quels produits achète-t-on **souvent avec** le vin ?* » — impossible en SQL.

Définition Data Mining

Ensemble de méthodes et techniques d'analyse de données et d'extraction d'informations structurées en vue d'aider à la prise de décision. Le « produit » du Data Mining, ce sont des **modèles** (les connaissances).

Données (information) $\xrightarrow{\text{Data Mining}}$ Modèles (connaissances)

Historique : **KDD** (1989, *Knowledge Discovery in Databases*), puis le terme **Data Mining** en 1991.

Attention : ce que le Data Mining n'est PAS

Ce n'est ni un entrepôt de données, ni du SQL, ni de l'OLAP, ni de la simple visualisation, ni de la statistique descriptive, et surtout **pas une boule de cristal** : il *extrapole* à partir de données existantes, il ne devine pas magiquement.

1.3 Les tâches du Data Mining

C'est le tableau le plus important du chapitre. Tout le reste du cours détaille ces tâches.

Tâche	Objectif	Méthodes	Appren
Classification	Attribuer une <i>classe connue</i> à un objet (crédit oui/non)	Arbres, K-NN, réseaux de neurones, Bayes	super
Prédiction	Estimer une <i>valeur future</i> d'un attribut	Arbres, réseaux de neurones	super
Association	Trouver les attributs <i>corrélés</i> (pânier : poisson & vin)	Apriori, FP-Growth	non sup
Segmentation	Former des <i>groupes homogènes</i> (classes inconnues)	K-means, K-NN, réseaux de neurones	non sup

Définition Supervisé vs non supervisé

Supervisé : l'apprenant reçoit des exemples avec *entrée ET sortie* (la classe est connue) → classification, prédiction.

Non supervisé : seulement des *entrées*, pas de classe → association, segmentation.

On distingue aussi les **modèles prédictifs** (extrapolent une info nouvelle : classification, prédiction) et les **modèles descriptifs** (mettent en évidence une info noyée : segmentation, association).

1.4 Le processus CRISP-DM

CRISP-DM (*Cross Industry Standard Process for Data Mining*) est la méthodologie la plus utilisée. Elle découpe un projet en **6 phases**, dans un cycle où l'on fait des « aller-retour ».

#	Phase	En quoi ça consiste
1	Compréhension du métier	Objectifs de l'entreprise → problème de data mining.
2	Compréhension des données	Recueillir, explorer, évaluer la qualité.
3	Préparation des données	Nettoyer, sélectionner, transformer (≈60 % du temps).
4	Modélisation	Choisir les algorithmes et calibrer leurs paramètres.
5	Évaluation	Les résultats répondent-ils aux objectifs ?
6	Déploiement	Mettre en œuvre les décisions (côté maître d'ouvrage).

Exemple — étude de cas Daimler-Chrysler

Objectif : **analyser les plaintes clients** dans l'automobile (réduire les coûts, améliorer la satisfaction). Les chercheurs ont suivi CRISP-DM : compréhension métier (« y a-t-il un lien entre plaintes ? »), exploration du système QUIS (7 millions de véhicules, 40 Go), préparation **très longue** (formats incompatibles), modélisation (réseaux bayésiens + règles d'association), évaluation **décevante** (modèles peu fiables à cause de la structure de la base), déploiement limité à un projet pilote. **Leçon** : **ne jamais sous-estimer la préparation des données.**

1.5 Outils et sources de données

Logiciels phares (sondages KDnuggets) : **RapidMiner**, **R**, **Excel**, **Weka**, **Python** (numpy/pandas/scikit-learn), **SAS**, **KNIME**... Dépôts de données pour s'entraîner : **UCI** (archive.ics.uci.edu), **Kaggle**, **Hugging Face**.

En résumé

Le Data Mining = extraire des **modèles** décisionnels des données. 4 tâches (classification, prédiction, association, segmentation), 2 régimes (supervisé / non supervisé), 1 méthodologie reine (**CRISP-DM**, 6 phases) où la **préparation des données** domine. Les chapitres suivants détaillent : la préparation (chap. 2), la classification (chap. 4) et la segmentation (chap. 5).

Prétraitement des données

Astuce / À retenir

Règle d'or du chapitre : « **Garbage in, garbage out** » — des données sales donnent des résultats faux. La préparation occupe **au moins 60 %** du temps d'un projet (Dorian Pyle).

2.1 Pourquoi prétraiter ?

Les données brutes sont souvent : **incomplètes** (valeurs manquantes), **bruitées** (erreurs, exceptions), **incohérentes** (nommage, codage), obsolètes ou mal formatées. Les 5 grandes étapes du prétraitement : **Nettoyage** → **Intégration** → **Transformation** → **Réduction** → **Discrétisation**.

2.2 Données manquantes

Causes : panne d'équipement, incohérence, non-saisie, oubli. Comment combler les « trous » ?

- **Ignorer** l'enregistrement (peu efficace si beaucoup de manquants) ;
- **Remplir manuellement** (laborieux) ;
- **Constante globale** (« inconnu ») ;
- **Moyenne** (variable numérique) ou **mode** (variable modale) ;
- **Moyenne par classe** (mieux) ;
- **Valeur la plus probable** (formule bayésienne / arbre de décision).

Calcul détaillé : combler un âge manquant de 3 façons

Soit la colonne Âge = [25, ?, 30, 40, 25] (une valeur manquante), avec une classe associée [A, A, B, B, A].

- **Moyenne globale** : sur les 4 valeurs connues, $\bar{x} = \frac{25+30+40+25}{4} = 30 \Rightarrow$ on met **30**.
- **Mode** (valeur la plus fréquente) : 25 apparaît 2 fois $\Rightarrow$ on met **25**.
- **Moyenne par classe** : le manquant est de classe « A » ; moyenne des âges de classe A connus = $\frac{25+25}{2} = 25 \Rightarrow$ on met **25** (plus « juste » car contextualisé).

2.3 Données bruitées

Bruit = erreur ou variance aléatoire d'une variable mesurée. On le corrige par : détection de points isolés, **clustering** (regrouper et repérer les isolés), inspection humaine, ou **régression** (lisser par une fonction).

2.4 Intégration & redondance

Intégration = combiner plusieurs sources en une seule. Problèmes : noms différents pour la même donnée (`num_client` $\equiv$ `client_id`), unités différentes (cm vs pouces). La **redondance** (un attribut déductible d'un autre) se détecte par **analyse de corrélation**.

2.5 Transformation des données

Lissage, agrégation, généralisation, et surtout les deux mises à l'échelle au programme :

Définition Normalisation min-max

Ramène chaque valeur dans l'intervalle $[0, 1]$:

$$X^* = \frac{X - \min(X)}{\max(X) - \min(X)}$$

La plus petite valeur devient 0, la plus grande devient 1.

Définition Standardisation (z-score)

Centre (moyenne 0) et réduit (écart-type 1) :

$$X^* = \frac{X - \bar{X}}{\sigma(X)}, \quad \bar{X} = \frac{1}{N} \sum_{i=1}^N x_i, \quad \sigma^2 = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{X})^2$$

Les valeurs tombent typiquement entre -3 et $+3$.

Calcul détaillé : min-max et z-score sur $X = \{200, 150, 300, 100, 250\}$

Min-max : $\min = 100$, $\max = 300$, étendue = 200.

$$\frac{200 - 100}{200} = 0.5; \quad \frac{150 - 100}{200} = 0.25; \quad \frac{300 - 100}{200} = 1; \quad \frac{100 - 100}{200} = 0; \quad \frac{250 - 100}{200} = 0.75$$

$\Rightarrow X^* = \{0.5, 0.25, 1, 0, 0.75\}$ — bien dans $[0, 1]$.

Z-score — étape 1, la *moyenne* : $\bar{X} = \frac{200 + 150 + 300 + 100 + 250}{5} = \frac{1000}{5} = 200$.

Étape 2, la *variance* (moyenne des carrés des écarts à la moyenne) :

$$\sigma^2 = \frac{1}{5} [(200-200)^2 + (150-200)^2 + (300-200)^2 + (100-200)^2 + (250-200)^2] = \frac{0+2500+10000+10000+2500}{5} = 5000,$$

donc l'écart-type $\sigma = \sqrt{5000} \approx 70,71$. Étape 3, on standardise chaque valeur :

$$\frac{200 - 200}{70,71} = 0; \quad \frac{150 - 200}{70,71} = -0,71; \quad \frac{300 - 200}{70,71} = +1,41; \quad \frac{100 - 200}{70,71} = -1,41; \quad \frac{250 - 200}{70,71} = +0,71$$

$\Rightarrow$ on vérifie : la nouvelle moyenne est ≈ 0 et le nouvel écart-type ≈ 1 .

2.6 Réduction des données

Obtenir une représentation **plus petite** mais aux résultats **quasi identiques** : agrégation par cubes, **réduction de dimension** (moins de colonnes/attributs), réduction de numérosité (moins de lignes), discrétisation.

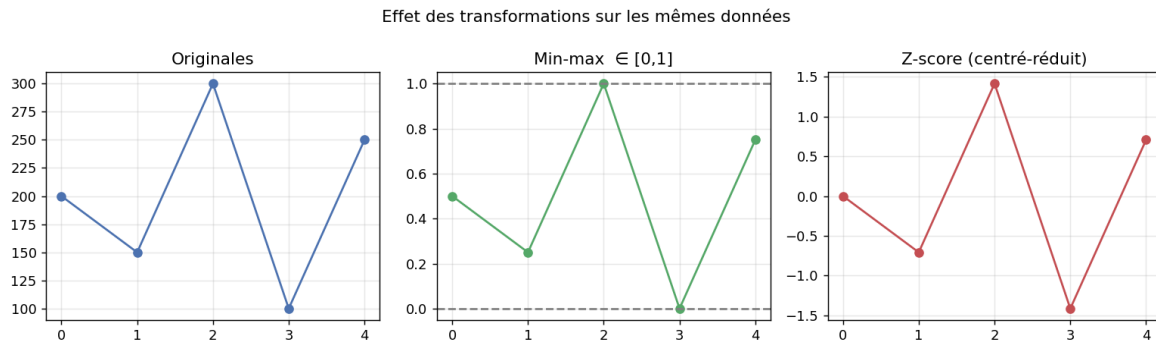


Fig. 2.1 : Mêmes données vues en brut, après min-max (dans $[0, 1]$) et après z-score (centré-réduit).

2.7 Exercices résolus

2.7.1 Exercice 1 — repérer les problèmes de qualité

ID	Code postal	Sexe	Revenu	Âge	État civil
1001	10048	M	75000	C	M
1002	J2S7K7	F	-40000	40	V
1003	90210	(vide)	10000000	45	C
1004	6269	M	50000	0	C
1005	55101	F	999999	30	D

Calcul détaillé : problèmes détectés

- **Valeur manquante** : sexe absent (ligne 1003).
- **Valeur hors domaine** : revenu = -40000 (négatif, impossible).
- **Valeur aberrante** : revenu = $10\,000\,000$ (extrême).
- **Code de manquant déguisé** : revenu = 999999 et âge = 0 (faux « vrais » chiffres).
- **Incohérence de type/format** : âge = « C » (non numérique) ; codes postaux mélangeant chiffres US (5 chiffres) et alphanumérique canadien (J2S7K7).

2.7.2 Exercice 2 — normalisation & standardisation (dataset churn)

La démarche (que le script automatise) :

1. vérifier les valeurs manquantes (`df.isna().sum()`) ;
2. comparer `area code` et `state` (incohérences géographiques) ;
3. transformer `day minutes` par **min-max** et vérifier sur une courbe que tout est dans $[0, 1]$;
4. transformer `night minutes` par **z-score** et lire l'intervalle obtenu (typiquement $[-3, +3]$).

2.8 Script Python commenté

Listing 2.1 : scripts/chap2_normalisation.py — min-max, z-score, qualité

```

1 """
2 CHAPITRE 2 - PRÉTRAITEMENT DES DONNÉES
3 =====
4 Ce script illustre, pas à pas, deux transformations du cours :
5 1. La NORMALISATION min-max      -> ramène les valeurs dans [0, 1]
6 2. La STANDARDISATION (z-score)   -> centre (moyenne 0) et réduit (écart-type 1)
7 Et il résout l'EXERCICE 1 (détection des problèmes de qualité d'un tableau).
8

```

```

9  On recode les formules "à la main" pour bien comprendre, puis on vérifie.
10 Les figures sont enregistrées dans rapport/figures/ pour le rapport LaTeX.
11 """
12
13 import os
14
15 import matplotlib
16
17 matplotlib.use("Agg") # backend sans écran : on enregistre les figures en PNG
18 import matplotlib.pyplot as plt
19 import numpy as np
20 import pandas as pd
21
22 FIG = os.path.join(os.path.dirname(__file__), "..", "rapport", "figures")
23 os.makedirs(FIG, exist_ok=True)
24
25
26 # -----
27 # 1. NORMALISATION MIN-MAX
28 # -----
29 # Formule du cours :  $X^* = (X - \min(X)) / (\max(X) - \min(X))$ 
30 # Après transformation, toutes les valeurs sont dans l'intervalle [0, 1].
31 def normalisation_min_max(x: np.ndarray) -> np.ndarray:
32     xmin, xmax = x.min(), x.max()
33     return (x - xmin) / (xmax - xmin)
34
35
36 # -----
37 # 2. STANDARDISATION (Z-SCORE)
38 # -----
39 # Formule du cours :  $X^* = (X - \text{moyenne}(X)) / \text{ecart\_type}(X)$ 
40 # moyenne :  $x_{\text{barre}} = (1/N) * \text{somme}(x_i)$ 
41 # variance :  $\sigma^2 = (1/N) * \text{somme}((x_i - x_{\text{barre}})^2)$  (écart-type = racine)
42 # Résultat : moyenne  $\sim 0$  et écart-type  $\sim 1$  (valeurs souvent entre -3 et +3).
43 def standardisation_zscore(x: np.ndarray) -> np.ndarray:
44     moyenne = x.mean()
45     ecart_type = x.std() # std de numpy = écart-type "population" (divise par N)
46     return (x - moyenne) / ecart_type
47
48
49 def demo_transformations() -> None:
50     print("=" * 70)
51     print("1) NORMALISATION MIN-MAX et 2) STANDARDISATION Z-SCORE")
52     print("=" * 70)
53
54     # Petit jeu de valeurs (ex. "day minutes" simplifié) pour voir les calculs
55     x = np.array([200.0, 150.0, 300.0, 100.0, 250.0])
56     print("Valeurs originales      :", x)
57
58     # --- min-max détaillé ---
59     xmin, xmax = x.min(), x.max()
60     print(f"\nmin(X) = {xmin}, max(X) = {xmax}, étendue = {xmax - xmin}")
61     xn = normalisation_min_max(x)
62     for xi, xni in zip(x, xn):
63         print(f" ({xi:6.1f} - {xmin}) / ({xmax} - {xmin}) = {xni:.3f}")
64     print("Normalisé (min-max)      :", np.round(xn, 3),
65           "\n-> bien dans [0, 1] :", xn.min() >= 0 and xn.max() <= 1)
66
67     # --- z-score détaillé ---
68     moyenne, sigma = x.mean(), x.std()
69     print(f"\nmoyenne = {moyenne}, écart-type (sigma) = {sigma:.3f}")
70     xs = standardisation_zscore(x)
71     for xi, xsi in zip(x, xs):
72         print(f" ({xi:6.1f} - {moyenne}) / {sigma:.3f} = {xsi:+.3f}")

```

```

73 print("Standardisé (z-score)      :", np.round(xs, 3))
74 print(f"    -> moyenne ≈ {xs.mean():.3f}, écart-type ≈ {xs.std():.3f}")
75
76 # --- figure comparative (comme demandé : "vérifier par une courbe") ---
77 fig, axes = plt.subplots(1, 3, figsize=(12, 3.6))
78 for ax, data, titre, col in zip(
79     axes,
80     [x, xn, xs],
81     ["Originales", "Min-max ∈ [0,1]", "Z-score (centré-réduit)"],
82     ["#4C72B0", "#55A868", "#C44E52"],
83 ):
84     ax.plot(data, "o-", color=col)
85     ax.set_title(titre)
86     ax.grid(alpha=0.3)
87 axes[1].axhline(0, ls="--", c="grey")
88 axes[1].axhline(1, ls="--", c="grey")
89 fig.suptitle("Effet des transformations sur les mêmes données")
90 fig.tight_layout()
91 fig.savefig(os.path.join(FIG, "chap2_transformations.png"), dpi=130)
92 print("\n[figure] rapport/figures/chap2_transformations.png enregistrée")
93
94
95 # -----
96 # EXERCICE 1 – Détecter les problèmes de qualité d'un tableau
97 # -----
98 def exercice1_qualite() -> None:
99     print("\n" + "=" * 70)
100    print("EXERCICE 1 – Problèmes de qualité des données")
101    print("=" * 70)
102    df = pd.DataFrame(
103        {
104            "ID_client": [1001, 1002, 1003, 1004, 1005],
105            "Code_postal": ["10048", "J2S7K7", "90210", "6269", "55101"],
106            "Sexe": ["M", "F", None, "M", "F"],
107            "Revenu": [75000, -40000, 10000000, 50000, 999999],
108            "Age": ["C", "40", "45", "0", "30"], # 'C' = valeur non numérique
109            "Etat_civil": ["M", "V", "C", "C", "D"],
110            "Transaction": [5000, 4000, 7000, 1000, 3000],
111        }
112    )
113    print(df.to_string(index=False))
114
115    print("\nProblèmes détectés automatiquement :")
116    # Sexe manquant
117    for i in df.index[df["Sexe"].isna()]:
118        print(f"    - Ligne {df.loc[i, 'ID_client']}: SEXE manquant (valeur nulle).")
119    # Revenu négatif ou aberrant
120    for i in df.index[df["Revenu"] < 0]:
121        print(f"    - Ligne {df.loc[i, 'ID_client']}: REVENU négatif ({df.loc[i, 'Revenu']}) -> impossible.")
122    for i in df.index[df["Revenu"] > 1_000_000]:
123        print(f"    - Ligne {df.loc[i, 'ID_client']}: REVENU aberrant ({df.loc[i, 'Revenu']}) -> valeur extrême.")
124    for i in df.index[df["Revenu"] == 999999]:
125        print(f"    - Ligne {df.loc[i, 'ID_client']}: REVENU = 999999 -> code 'valeur manquante' déguisé.")
126    # Age non numérique ou impossible
127    for i in df.index:
128        age = df.loc[i, "Age"]
129        if not str(age).isdigit():
130            print(f"    - Ligne {df.loc[i, 'ID_client']}: AGE='{age}' non numérique (incohérence de type).")
131        elif int(age) == 0:
132            print(f"    - Ligne {df.loc[i, 'ID_client']}: AGE=0 -> probablement manquant/erronné.")

```

```
133 # Code postal incohérent (format mixte chiffres / lettres, longueurs variables)
134 for i in df.index:
135     cp = df.loc[i, "Code_postal"]
136     if not cp.isdigit():
137         print(f" - Ligne {df.loc[i, 'ID_client']}: CODE_POSTAL='{cp}' format non numérique (US
138 vs Canada).")
139     elif len(cp) != 5:
140         print(f" - Ligne {df.loc[i, 'ID_client']}: CODE_POSTAL='{cp}' longueur ≠ 5
141 (incohérent).")
142
143 print(
144     "\nRésumé : valeurs manquantes (sexe), valeurs hors domaine (revenu négatif),\n"
145     "valeurs aberrantes/extrêmes (10 000 000), codes de manquant déguisés (999999, age 0),\n"
146     "et incohérences de format/codage (codes postaux US/Canada mélangés).")
147 )
148 if __name__ == "__main__":
149     demo_transformations()
150     exercicel_qualite()
```

En résumé

Prétraiter = nettoyer (manquants, bruit), intégrer (sources, redondance), transformer (**min-max** dans $[0, 1]$, **z-score** centré-réduit) et réduire. C'est l'étape la plus chronophage mais elle conditionne tout le reste — en particulier le K-NN et le k-means qui dépendent des distances.

Classification

Définition Classification

Raisonnement à **partir de cas** : on a des enregistrements décrits par des attributs, dont l'un est la **classe**. On cherche un **modèle** qui exprime la classe en fonction des autres attributs, pour ensuite **prédire la classe** d'enregistrements inconnus. C'est de l'apprentissage **supervisé**.

Deux étapes : (1) *construction du modèle* sur l'ensemble d'**apprentissage** (*training set*, $\approx 2/3$) ; (2) *utilisation/test* du modèle sur l'ensemble de **test** ($\approx 1/3$) pour mesurer la précision. On peut aussi faire de la **validation croisée** : découper en k paquets, apprendre sur $k-1$ et tester sur le dernier, en tournant.

3.1 Méthode des K plus proches voisins (K-NN)

Définition K-NN

Pas d'apprentissage : le modèle *est* l'échantillon. Pour classer un nouvel individu y : on calcule sa **distance** à tous les individus connus, on garde les K **plus proches**, et on attribue à y la **classe majoritaire** parmi ces K voisins. Heuristique : $K = \text{nombre d'attributs} + 1$.

Vote simple : chaque voisin compte pour 1. **Vote pondéré** : chaque voisin a un poids $\propto 1/\text{distance}$ (plus il est proche, plus il pèse). **Confiance** = votes gagnants / total des votes.

Distances entre numériques ($x, y \in \mathbb{R}^m$) :

$$d_{\text{Euclidienne}} = \sqrt{\sum_i (x_i - y_i)^2}, \quad d_{\text{Manhattan}} = \sum_i |x_i - y_i|, \quad d_{\text{Minkowski}} = \left(\sum_i |x_i - y_i|^q \right)^{1/q}.$$

($q = 1$: Manhattan ; $q = 2$: Euclidienne.) **Données nominales** : $d = 0$ si égales, 1 sinon.

Attention

K-NN dépend des distances : il faut **normaliser** les attributs avant, sinon l'attribut aux grandes valeurs (ex. le salaire) écrase les autres.

3.1.1 Exercice résolu — classer David ($K = 3$, euclidienne)

Client	Âge	Salaire	Nb cartes	Classe (Loyal)
Jean	35	35000	3	N
Rachèle	22	50000	2	O
Anne	63	200000	1	N
Thomas	59	170000	1	N
Nathalie	25	40000	4	O
David	37	50000	2	?

Démarche, en 3 temps : (1) on calcule la distance de David à *chacun* des 5 clients connus ; (2) on *trie* ces distances et on garde les $K = 3$ plus petites ; (3) on regarde la classe de ces 3 voisins et on *vote*.

Calcul détaillé : distances euclidiennes de David (37; 50000; 2) à chacun

On applique $d = \sqrt{(\Delta\text{âge})^2 + (\Delta\text{salaire})^2 + (\Delta\text{cartes})^2}$. On écrit *tous* les termes avant de simplifier :

$$d(\text{David, Jean}) = \sqrt{(37 - 35)^2 + (50000 - 35000)^2 + (2 - 3)^2} = \sqrt{\underbrace{4}_{\text{âge}} + \underbrace{225\,000\,000}_{\text{salaire}} + \underbrace{1}_{\text{cartes}}} \approx \mathbf{15000,0}$$

$$d(\text{David, Rachèle}) = \sqrt{(37 - 22)^2 + (50000 - 50000)^2 + (2 - 2)^2} = \sqrt{225 + 0 + 0} = \mathbf{15,0}$$

$$d(\text{David, Anne}) = \sqrt{(37 - 63)^2 + (50000 - 200000)^2 + (2 - 1)^2} = \sqrt{676 + 22\,500\,000\,000 + 1} \approx \mathbf{150000,0}$$

$$d(\text{David, Thomas}) = \sqrt{(37 - 59)^2 + (50000 - 170000)^2 + (2 - 1)^2} = \sqrt{484 + 14\,400\,000\,000 + 1} \approx \mathbf{120000,0}$$

$$d(\text{David, Nathalie}) = \sqrt{(37 - 25)^2 + (50000 - 40000)^2 + (2 - 4)^2} = \sqrt{144 + 100\,000\,000 + 4} \approx \mathbf{10000,0}$$

Tri croissant : Rachèle 15 < Nathalie 10 000 < Jean 15 000 < Thomas 120 000 < Anne 150 000.

Les $K = 3$ plus proches : Rachèle (**O**), Nathalie (**O**), Jean (**N**).

Calcul détaillé : vote simple — détail

On compte les classes des 3 voisins : « O » apparaît 2 fois (Rachèle, Nathalie), « N » 1 fois (Jean).

$$\text{score}(O) = 2, \quad \text{score}(N) = 1 \Rightarrow \text{classe majoritaire} = \mathbf{O}.$$

$$\text{Confiance} = \frac{\text{voix gagnantes}}{\text{total des voix}} = \frac{2}{3} \approx \mathbf{67\%}.$$

Calcul détaillé : vote pondéré — détail

Chaque voisin pèse $w = 1/d$ (proche $\Rightarrow$ gros poids). On additionne les poids *par classe* :

$$w_O = \underbrace{\frac{1}{15}}_{\text{Rachèle}} + \underbrace{\frac{1}{10000}}_{\text{Nathalie}} \approx 0,06667 + 0,00010 = 0,06677, \quad w_N = \underbrace{\frac{1}{15000}}_{\text{Jean}} \approx 0,00007.$$

Comme $w_O \gg w_N$, la classe pondérée est **O**. Confiance pondérée = $\frac{w_O}{w_O + w_N} = \frac{0,06677}{0,06684} \approx \mathbf{99,9\%}$. **Conclusion identique** : David est classé **O** (Loyal = O).

Attention : le piège des échelles

Regardez les termes sous la racine : le salaire pèse $\sim 10^8$, l'âge $\sim 10^2$, les cartes ~ 1 . **Le salaire écrase tout** : la distance est, à 0,001 % près, égale à l'écart de salaire. On ne fait donc, en réalité, du K-NN *que sur le salaire*. C'est pourquoi le cours recommande de **normaliser avant un K-NN**.

Calcul détaillé : la même classification, mais normalisée (min-max)

On ramène chaque attribut dans $[0, 1]$ avec $x^* = \frac{x - \min}{\max - \min}$, à partir des bornes de l'apprentissage : âge $\in [22, 63]$ (étendue 41), salaire $\in [35000, 200000]$ (étendue 165000), cartes $\in [1, 4]$ (étendue 3).

David $\rightarrow (\frac{37-22}{41}; \frac{50000-35000}{165000}; \frac{2-1}{3}) = (0,366; 0,091; 0,333)$. Distances normalisées :

	Jean	Rachèle	Anne	Thomas	Nathalie
distance	0,349 (N)	0,366 (O)	1,157 (N)	0,963 (N)	0,731 (O)

Surprise : maintenant que les 3 attributs comptent autant, **Jean (N) devient le plus proche** ! Mais les 3 plus proches sont Jean (N), Rachèle (O), Nathalie (O) : le vote donne encore **O** (2 contre 1). La normalisation a changé les distances et le classement, **pas la décision finale** ici.

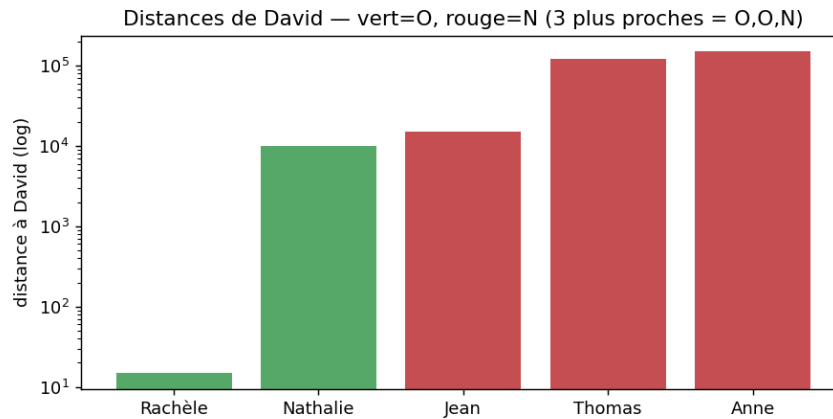


Fig. 3.1 : Distances brutes de David (échelle log à cause du salaire). Vert = O, rouge = N. Les 3 plus proches : O, O, N $\Rightarrow$ O.

Listing 3.1 : scripts/knn_david.py

```

1  """
2  CHAPITRE 4 – CLASSIFICATION : méthode des K plus proches voisins (K-NN)
3  =====
4  EXERCICE : classer le client "David" dans la classe N ou O, avec K = 3
5              et la distance euclidienne (vote simple ET vote pondéré).
6
7  Données (Age, Salaire, Nb cartes de crédit, classe "Loyal") :
8      Jean    35    35000    3    N
9      Rachèle  22    50000    2    O
10     Anne    63    200000    1    N
11     Thomas  59    170000    1    N
12     Nathalie 25    40000    4    O
13     David   37    50000    2    ?    <-- à classer
14
15  On code la distance euclidienne à la main, puis on applique :
16      - vote simple      : la classe majoritaire parmi les K voisins
17      - vote pondéré     : chaque voisin a un poids = 1 / distance
18  La "confiance" = votes gagnants / total des votes.
19  """
20
21  import os
22
23  import matplotlib
24
25  matplotlib.use("Agg")
26  import matplotlib.pyplot as plt
27
28  FIG = os.path.join(os.path.dirname(__file__), "..", "rapport", "figures")
29  os.makedirs(FIG, exist_ok=True)
30
31  # (nom, age, salaire, nb_cartes, classe)
32  APPRENTISSAGE = [
33      ("Jean", 35, 35000, 3, "N"),
34      ("Rachèle", 22, 50000, 2, "O"),
35      ("Anne", 63, 200000, 1, "N"),
36      ("Thomas", 59, 170000, 1, "N"),

```

```

37     ("Nathalie", 25, 40000, 4, "0"),
38 ]
39 DAVID = (37, 50000, 2)
40 K = 3
41
42
43 def distance_euclidienne(a, b) -> float:
44     """racine( somme (a_i - b_i)^2 ) sur toutes les dimensions."""
45     return sum((ai - bi) ** 2 for ai, bi in zip(a, b)) ** 0.5
46
47
48 def knn(point, donnees, k):
49     # 1) distance de 'point' à chaque individu connu
50     distances = []
51     for nom, age, sal, cartes, classe in donnees:
52         d = distance_euclidienne(point, (age, sal, cartes))
53         distances.append((nom, classe, d))
54     # 2) tri par distance croissante, on garde les k plus proches
55     distances.sort(key=lambda t: t[2])
56     voisins = distances[:k]
57     return distances, voisins
58
59
60 def vote_simple(voisins):
61     scores = {}
62     for _, classe, _ in voisins:
63         scores[classe] = scores.get(classe, 0) + 1
64     gagnant = max(scores, key=scores.get)
65     confiance = scores[gagnant] / sum(scores.values())
66     return gagnant, scores, confiance
67
68
69 def vote_pondere(voisins):
70     # poids = 1/distance : plus un voisin est proche, plus il pèse lourd
71     scores = {}
72     for _, classe, d in voisins:
73         scores[classe] = scores.get(classe, 0.0) + 1.0 / d
74     gagnant = max(scores, key=scores.get)
75     confiance = scores[gagnant] / sum(scores.values())
76     return gagnant, scores, confiance
77
78
79 def normaliser_min_max(donnees, point):
80     """Ramène chaque attribut dans [0,1] avec les min/max de l'apprentissage.
81     Renvoie (donnees_normalisees, point_normalise)."""
82     # min et max de chaque colonne (age, salaire, cartes) sur l'apprentissage
83     colonnes = list(zip(*[(a, s, c) for _, a, s, c, _ in donnees]))
84     mins = [min(col) for col in colonnes]
85     maxs = [max(col) for col in colonnes]
86
87     def norm(triplet):
88         return tuple((v - mn) / (mx - mn) for v, mn, mx in zip(triplet, mins, maxs))
89
90     dn = [(nom, *norm((a, s, c)), cl) for nom, a, s, c, cl in donnees]
91     pn = norm(point)
92     return dn, pn, mins, maxs
93
94
95 def main():
96     print("=" * 70)
97     print(f"K-NN — classer David {DAVID} avec K={K} (distance euclidienne)")
98     print("=" * 70)
99
100     distances, voisins = knn(DAVID, APPRENTISSAGE, K)

```

```

101
102 print("\nDistances de David à chaque individu (détail du calcul) :")
103 for nom, age, sal, cartes, classe in APPRENTISSAGE:
104     d = distance_euclidienne(DAVID, (age, sal, cartes))
105     print(
106         f"    d(David,{nom:8s}) = racine(({DAVID[0]}-{age})2 + "
107         f"    f"({DAVID[1]}-{sal})2 + ({DAVID[2]}-{cartes})2) = {d:12.3f}    [{classe}]"
108     )
109
110 print(f"\nLes K={K} plus proches voisins :")
111 for nom, classe, d in voisins:
112     print(f"    {nom:8s}    classe={classe}    distance={d:.3f}")
113
114 g1, s1, c1 = vote_simple(voisins)
115 print(f"\nVOTE SIMPLE      : {s1} -> classe = {g1}    (confiance {c1:.0%})")
116
117 g2, s2, c2 = vote_pondere(voisins)
118 s2r = {k: round(v, 6) for k, v in s2.items()}
119 print(f"VOTE PONDÉRÉ      : {s2r} -> classe = {g2}    (confiance {c2:.0%})")
120
121 print(
122     "\nConclusion : David est classé '0' (Loyal=0) dans les deux votes.\n"
123     "Remarque importante du cours : le salaire ≈( dizaines de milliers) écrase\n"
124     "l'âge et le nombre de cartes. D'où la recommandation de NORMALISER avant\n"
125     "un K-NN pour que chaque attribut compte autant."
126 )
127
128 # -----
129 # VERSION NORMALISÉE (min-max) – recommandée par le cours
130 # -----
131 print("\n" + "-" * 70)
132 print("VERSION NORMALISÉE (min-max) : chaque attribut compte autant")
133 print("-" * 70)
134 dn, pn, mins, maxs = normaliser_min_max(APPRENTISSAGE, DAVID)
135 print(f"min par colonne = {mins}, max = {maxs}")
136 print(f"David normalisé = ({pn[0]:.3f}, {pn[1]:.3f}, {pn[2]:.3f})")
137 dist_n = []
138 for nom, a, s, c, classe in dn:
139     d = distance_euclidienne(pn, (a, s, c))
140     dist_n.append((nom, classe, d))
141     print(f"    d(David,{nom:8s}) = {d:.3f}    [{classe}]    "
142         f"    f"normalisé=({a:.3f},{s:.3f},{c:.3f})")
143 dist_n.sort(key=lambda t: t[2])
144 voisins_n = dist_n[:K]
145 gn, sn, cn = vote_simple(voisins_n)
146 print(f"    3 plus proches normalisés : {[ (n, round(d,3)) for n,_,d in voisins_n ]}")
147 print(f"    -> vote simple = {gn} (confiance {cn:.0%})")
148 print("    Remarque : après normalisation, Jean (N) devient le PLUS proche,\n"
149     "    mais '0' l'emporte quand même 2 voix contre 1.")
150
151 # Vérification avec scikit-learn
152 try:
153     from sklearn.neighbors import KNeighborsClassifier
154
155     X = [[a, s, c] for _, a, s, c, _ in APPRENTISSAGE]
156     y = [cl for _, cl in APPRENTISSAGE]
157     clf = KNeighborsClassifier(n_neighbors=K).fit(X, y)
158     print(f"\n[sklearn] prédiction = {clf.predict([list(DAVID)])[0]}")
159 except Exception as e: # pragma: no cover
160     print("sklearn indisponible:", e)
161
162 # Figure : barres des distances (échelle log car salaire domine)
163 noms = [t[0] for t in distances]
164 ds = [t[2] for t in distances]

```

```

165 cols = ["#55A868" if t[1] == "0" else "#C44E52" for t in distances]
166 fig, ax = plt.subplots(figsize=(7, 3.6))
167 ax.bar(noms, ds, color=cols)
168 ax.set_yscale("log")
169 ax.set_ylabel("distance à David (log)")
170 ax.set_title("Distances de David - vert=0, rouge=N (3 plus proches = 0,0,N)")
171 fig.tight_layout()
172 fig.savefig(os.path.join(FIG, "knn_distances.png"), dpi=130)
173 print("[figure] rapport/figures/knn_distances.png enregistrée")
174
175
176 if __name__ == "__main__":
177     main()

```

3.2 Arbres de décision

Définition Arbre de décision

Représentation graphique d'une procédure de classification. **Nœud interne** = test sur un attribut ; **branche** = résultat du test ; **feuille** = classe. Chaque chemin racine→feuille donne une **règle SI...ALORS**.

Construction (récursive) : (1) toutes les instances à la racine ; (2) choisir le **meilleur attribut** de découpage (selon un critère) ; (3) partitionner ; (4) recommencer sur chaque fils ; (5) arrêter quand le nœud est **pur** (une seule classe), presque pur, ou qu'il n'y a plus d'attribut. Critères de choix : **indice de Gini** (CART), **gain d'information** (ID3, C4.5), χ^2 (CHAID).

3.2.1 Critère 1 — Indice de Gini

Définition Gini

$$GINI(t) = 1 - \sum_j [p(j|t)]^2$$

$p(j|t)$ = fréquence de la classe j au nœud t . **Gini** = 0 si le nœud est **pur** ; plus Gini est **bas**, plus le nœud est **pur**. Pour un découpage en k fils :

$$GINI_{\text{split}} = \sum_{i=1}^k \frac{n_i}{n} GINI(i) \quad (\text{à minimiser}).$$

Calcul détaillé : petits exemples de Gini (sur 6 enregistrements)

$(C_1=0, C_2=6) : GINI = 1 - 0^2 - 1^2 = \mathbf{0}$ (pur). $(C_1=1, C_2=5) : GINI = 1 - (\frac{1}{6})^2 - (\frac{5}{6})^2 = \mathbf{0,278}$. $(C_1=2, C_2=4) : GINI = 1 - (\frac{2}{6})^2 - (\frac{4}{6})^2 = \mathbf{0,444}$.

Exercice résolu (assurance auto).

Âge	Type_voit	Risque
23	Familiale	Élevé
17	Sport	Élevé
43	Sport	Élevé
68	Familiale	Faible
32	Camion	Faible
20	Familiale	Élevé

Plan : on évalue chaque attribut comme candidat de découpage de la racine, on garde celui qui donne le plus petit $GINI_{split}$, puis on recommence sur chaque fils.

Calcul détaillé : étape 0 — impureté de la racine

La racine contient les 6 enregistrements : 4 « Élevé » et 2 « Faible ». Les fréquences sont $p(\text{Élevé}) = \frac{4}{6}$ et $p(\text{Faible}) = \frac{2}{6}$, donc

$$GINI(\text{racine}) = 1 - \left(\frac{4}{6}\right)^2 - \left(\frac{2}{6}\right)^2 = 1 - \frac{16}{36} - \frac{4}{36} = 1 - 0,444 - 0,111 = \mathbf{0,444}.$$

But : trouver le découpage qui fait *le plus baisser* cette valeur.

Calcul détaillé : candidat A — l'attribut nominal Type_voit

Découpage multi-branche, un fils par valeur :

- **Familiale** = {Élevé (23), Faible (68), Élevé (20)} : 2 Élevé, 1 Faible, donc $GINI = 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = 1 - 0,444 - 0,111 = 0,444$;
- **Sport** = {Élevé (17), Élevé (43)} : *pur* $\Rightarrow GINI = 0$;
- **Camion** = {Faible (32)} : *pur* $\Rightarrow GINI = 0$.

Moyenne pondérée par la taille des fils (3, 2, 1 sur 6) :

$$GINI_{split} = \frac{3}{6}(0,444) + \frac{2}{6}(0) + \frac{1}{6}(0) = \mathbf{0,222}.$$

Calcul détaillé : candidat B — l'attribut continu Âge (on teste tous les seuils)

On trie les âges et leur classe : 17_É, 20_É, 23_É, 32_F, 43_É, 68_F. Un seuil candidat est le *milieu* de deux âges consécutifs.

Seuil	Gauche (Âge < seuil)	Droite (Âge ≥ seuil)	$GINI_{split}$
18,5	{17} : É — pur, 0	{20, 23, 32, 43, 68} : 3É, 2F, 0,48	$\frac{1}{6}(0) + \frac{5}{6}(0,48) = 0,400$
21,5	{17, 20} : pur, 0	{23, 32, 43, 68} : 2É, 2F, 0,50	$\frac{2}{6}(0) + \frac{4}{6}(0,50) = 0,333$
27,5	{17, 20, 23} : pur, 0	{32, 43, 68} : 1É, 2F, 0,444	$\frac{3}{6}(0) + \frac{3}{6}(0,444) = \mathbf{0,222}$
37,5	{17, 20, 23, 32} : 3É, 1F, 0,375	{43, 68} : 1É, 1F, 0,50	$\frac{4}{6}(0,375) + \frac{2}{6}(0,50) = 0,417$
55,5	{17, 20, 23, 32, 43} : 4É, 1F, 0,32	{68} : pur, 0	$\frac{5}{6}(0,32) + \frac{1}{6}(0) = 0,267$

Le **meilleur seuil** est Âge < 27,5 avec $GINI_{split} = \mathbf{0,222}$ (détail d'un $GINI$ de fils : pour la droite {32, 43, 68}, 1 Élevé et 2 Faible donnent $1 - \left(\frac{1}{3}\right)^2 - \left(\frac{2}{3}\right)^2 = 0,444$).

Astuce / À retenir : pourquoi Âge < 27,5 et pas Type_voit ?

Les deux donnent $GINI_{split} = 0,222$: **égalité parfaite**. En cas d'égalité, CART/scikit-learn tranche selon l'ordre interne des variables et choisit ici **Âge** < 27,5. On suit ce choix (l'autre arbre, démarrant par **Type_voit**, serait tout aussi correct).

Calcul détaillé : 2^e niveau — le fils « Âge ≥ 27,5 »

Le fils gauche (« Âge < 27,5 ») est *pur* (tous Élevé) : c'est une feuille, on s'arrête. Le fils droit contient {32_{Camion,F}, 43_{Sport,É}, 68_{Familiale,F}}, soit 1 Élevé et 2 Faible : $GINI = 0,444$, il faut le re-découper.

- **Par Type_voit** : Sport → {É} pur, Familiale → {F} pur, Camion → {F} pur. $GINI_{split} = 0$ — **séparation parfaite**.

- **Par Âge** (seuils 37,5 et 55,5) : on retombe à chaque fois sur $GINI_{split} = \frac{1}{3}(0) + \frac{2}{3}(0,5) = 0,333$.
 $0 < 0,333$: on choisit donc **Type_voit**, qui classe parfaitement les 3 derniers cas.

Exemple — arbre Gini final

```

Âge < 27,5 ?
|-- OUI   > Élevé
|-- NON   > Type_voit ?
|-- Sport > Élevé
|-- Familiale > Faible
|-- Camion > Faible

```

Arbre de décision — Indice de Gini (scikit-learn)

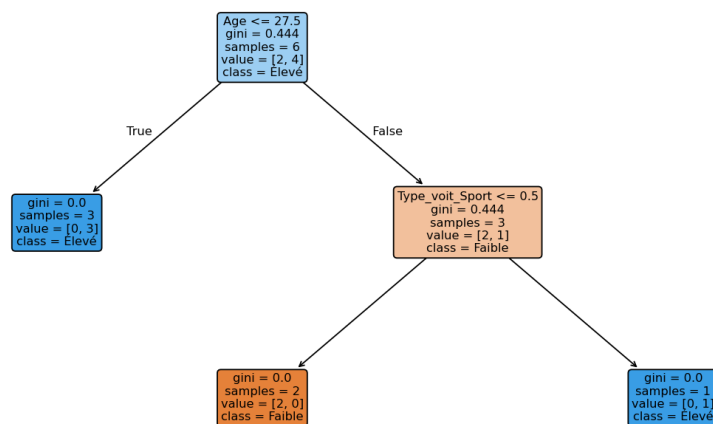


Fig. 3.2 : Arbre de décision Gini reconstruit par scikit-learn (**Type_voit** encodé en one-hot).

Listing 3.2 : scripts/gini_arbre.py

```

1  """
2  CHAPITRE 4 — ARBRE DE DÉCISION par l'INDICE DE GINI (CART)
3  =====
4  Données d'apprentissage (assurance auto) :
5  ID  Age  Type_voit  Risque
6  0   23  Familiale  Élevé
7  1   17  Sport      Élevé
8  2   43  Sport      Élevé
9  3   68  Familiale  Faible
10  4   32  Camion     Faible
11  5   20  Familiale  Élevé
12
13  RAPPELS (cours) :
14  Gini(t) = 1 - somme_j p(j|t)^2          (θ = 0 : pur, plus bas = plus pur)
15  Gini_split = somme_i (n_i / n) * Gini(i)  (moyenne pondérée des enfants)
16  -> on choisit le découpage qui MINIMISE Gini_split.
17
18  On code tout à la main pour voir chaque calcul, puis on vérifie avec scikit-learn.
19  """
20
21  import os
22

```

```

23 import matplotlib
24
25 matplotlib.use("Agg")
26 import matplotlib.pyplot as plt
27 import pandas as pd
28
29 FIG = os.path.join(os.path.dirname(__file__), "..", "rapport", "figures")
30 os.makedirs(FIG, exist_ok=True)
31
32 DATA = pd.DataFrame(
33     {
34         "Age": [23, 17, 43, 68, 32, 20],
35         "Type_voit": ["Familiale", "Sport", "Sport", "Familiale", "Camion", "Familiale"],
36         "Risque": ["Élevé", "Élevé", "Élevé", "Faible", "Faible", "Élevé"],
37     }
38 )
39
40
41 def gini(labels) -> float:
42     """1 - somme des p^2. labels = liste/série de classes."""
43     n = len(labels)
44     if n == 0:
45         return 0.0
46     g = 1.0
47     for c in set(labels):
48         p = sum(1 for x in labels if x == c) / n
49         g -= p**2
50     return g
51
52
53 def gini_split(groupe) -> float:
54     """Moyenne pondérée des Gini des groupes (enfants)."""
55     n = sum(len(g) for g in groupe)
56     return sum(len(g) / n * gini(g) for g in groupe)
57
58
59 def evaluer_attribut_nominal(df, attr, cible):
60     """Découpage multi-branche : un enfant par valeur de l'attribut."""
61     groupe = [g[cible].tolist() for _, g in df.groupby(attr)]
62     return gini_split(groupe), {v: g[cible].tolist() for v, g in df.groupby(attr)}
63
64
65 def evaluer_attribut_continu(df, attr, cible):
66     """Découpage binaire (attr < seuil). On teste tous les milieux de valeurs."""
67     vals = sorted(df[attr].unique())
68     meilleurs = (1.0, None, None)
69     details = []
70     for i in range(len(vals) - 1):
71         seuil = (vals[i] + vals[i + 1]) / 2
72         gauche = df[df[attr] < seuil][cible].tolist()
73         droite = df[df[attr] >= seuil][cible].tolist()
74         gs = gini_split([gauche, droite])
75         details.append((seuil, gauche, droite, gs))
76         if gs < meilleurs[0]:
77             meilleurs = (gs, seuil, (gauche, droite))
78     return meilleurs, details
79
80
81 def main():
82     print("=" * 70)
83     print("ARBRE DE DÉCISION – INDICE DE GINI")
84     print("=" * 70)
85     print(DATA.to_string(index=False))
86

```

```

87  racine = DATA["Risque"].tolist()
88  print(f"\nGini(racine) = {gini(racine):.4f}  "
89        f"(Élevé={racine.count('Élevé')}, Faible={racine.count('Faible')})")
90
91  # --- Attribut Type_voit (nominal) ---
92  gs_type, groupes = evaluer_attribut_nominal(DATA, "Type_voit", "Risque")
93  print("\n[Type_voit] (multi-branche)")
94  for v, lab in groupes.items():
95      print(f"    {v:10s} {lab}  Gini={gini(lab):.4f}")
96  print(f"    => Gini_split(Type_voit) = {gs_type:.4f}")
97
98  # --- Attribut Age (continu) ---
99  print("\n[Age] (binaire, test de tous les seuils)")
100 (gs_age, seuil, _), details = evaluer_attribut_continu(DATA, "Age", "Risque")
101 for s, gch, dr, gsv in details:
102     print(f"    Age < {s:5.1f} : gauche={gch} droite={dr}  Gini_split={gsv:.4f}")
103 print(f"    => meilleur seuil Age < {seuil}  Gini_split = {gs_age:.4f}")
104
105 print("\nComparaison du 1er découpage :")
106 print(f"    Type_voit -> {gs_type:.4f} | Age<{seuil} -> {gs_age:.4f}")
107 print("    (égalité ! scikit-learn choisit Age<27.5, on suit ce choix)\n")
108
109 print("ARBRE OBTENU :")
110 print("    Age < 27.5 ?")
111 print("    ├── OUI -> Élevé (feuille pure : ages 17,20,23 tous Élevé)")
112 print("    └── NON -> Type_voit ?")
113 print("        ├── Sport -> Élevé")
114 print("        ├── Familiale -> Faible")
115 print("        └── Camion -> Faible")
116
117 # --- Vérification scikit-learn + figure ---
118 from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
119
120 enc = pd.get_dummies(DATA, columns=["Type_voit"])
121 X = enc.drop("Risque", axis=1)
122 y = enc["Risque"]
123 model = DecisionTreeClassifier(criterion="gini", random_state=0).fit(X, y)
124 print("\n[sklearn] règles de l'arbre :")
125 print(export_text(model, feature_names=list(X.columns)))
126
127 plt.figure(figsize=(11, 6))
128 plot_tree(model, feature_names=list(X.columns),
129           class_names=model.classes_, filled=True, rounded=True, fontsize=9)
130 plt.title("Arbre de décision – Indice de Gini (scikit-learn)")
131 plt.tight_layout()
132 plt.savefig(os.path.join(FIG, "gini_arbre.png"), dpi=130)
133 print("[figure] rapport/figures/gini_arbre.png enregistrée")
134
135
136 if __name__ == "__main__":
137     main()

```

3.2.2 Critère 2 — Gain d'information (entropie)

Définition Entropie et gain

$$Entropie(t) = - \sum_j p(j|t) \log_2 p(j|t), \quad GAIN_{\text{split}} = Entropie(p) - \sum_{i=1}^k \frac{n_i}{n} Entropie(i).$$

Entropie = 0 si pur, maximale ($\log_2 n_c$) si classes équilibrées. On choisit l'attribut qui **maximise le gain** (la plus grande réduction d'entropie). **C4.5** corrige en divisant par le *SplitINFO*

(*gain ratio*) pour pénaliser les attributs à trop de valeurs.

Calcul détaillé : comprendre l'entropie sur 3 cas (6 enregistrements)

On note $\log_2 x = \frac{\ln x}{\ln 2}$. Plus le mélange est équilibré, plus l'entropie est grande.

- ($C_1=0, C_2=6$) : tout dans une classe. $-\frac{0}{6} \log_2 \frac{0}{6} - \frac{6}{6} \log_2 \frac{6}{6} = -0 - 1 \cdot \log_2 1 = -0 - 0 = 0$ (nœud *pur*).
- ($C_1=1, C_2=5$) : $-\frac{1}{6} \log_2 \frac{1}{6} - \frac{5}{6} \log_2 \frac{5}{6} = -\frac{1}{6}(-2,585) - \frac{5}{6}(-0,263) = 0,431 + 0,219 = \mathbf{0,650}$.
- ($C_1=2, C_2=4$) : $-\frac{2}{6} \log_2 \frac{2}{6} - \frac{4}{6} \log_2 \frac{4}{6} = -\frac{1}{3}(-1,585) - \frac{2}{3}(-0,585) = 0,528 + 0,390 = \mathbf{0,918}$.

Exercice résolu (achat d'ordinateur, 14 individus). C'est l'exemple du TP. Classe Achète_ordinateur : 9 « oui » et 5 « non ».

Calcul détaillé : étape 0 — entropie de la racine

$$Entropie(S) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = -0,643 \cdot (-0,637) - 0,357 \cdot (-1,485) = 0,410 + 0,530 = \mathbf{0,940}.$$

Calcul détaillé : gain de l'attribut Âge

Âge a 3 valeurs. On compte les classes dans chaque groupe, puis son entropie :

- ≤ 30 : 5 individus, **2 oui / 3 non** $\Rightarrow -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0,529 + 0,442 = 0,971$;
- 31..40 : 4 individus, **4 oui / 0 non** $\Rightarrow$ pur, entropie = 0 ;
- > 40 : 5 individus, **3 oui / 2 non** $\Rightarrow 0,971$ (symétrique du premier).

Entropie restante (pondérée par 5, 4, 5 sur 14) :

$$E_{\text{reste}} = \frac{5}{14}(0,971) + \frac{4}{14}(0) + \frac{5}{14}(0,971) = 0,694 \Rightarrow \boxed{GAIN(\hat{\text{Age}}) = 0,940 - 0,694 = 0,247}.$$

Calcul détaillé : gain des trois autres attributs

Revenue (élevé : 2o/2n $\rightarrow 1,0$; moyen : 4o/2n $\rightarrow 0,918$; bas : 3o/1n $\rightarrow 0,811$) :

$$E_{\text{reste}} = \frac{4}{14}(1) + \frac{6}{14}(0,918) + \frac{4}{14}(0,811) = 0,911 \Rightarrow GAIN = \mathbf{0,029}.$$

Étudiant (oui : 6o/1n $\rightarrow 0,592$; non : 3o/4n $\rightarrow 0,985$) :

$$E_{\text{reste}} = \frac{7}{14}(0,592) + \frac{7}{14}(0,985) = 0,789 \Rightarrow GAIN = \mathbf{0,151}.$$

État_civil (célibataire : 6o/2n $\rightarrow 0,811$; marié : 3o/3n $\rightarrow 1,0$) :

$$E_{\text{reste}} = \frac{8}{14}(0,811) + \frac{6}{14}(1) = 0,892 \Rightarrow GAIN = \mathbf{0,048}.$$

Astuce / À retenir : choix de la racine

Attribut	Âge	Étudiant	État_civil	Revenue
Gain	0,247	0,151	0,048	0,029

Âge a le **plus grand gain** (réduit le plus l'entropie) : c'est la **racine**.

Calcul détaillé : 2^e niveau — re-découper chaque fils de l'âge

Âge = 31..40 : déjà pur (4 oui) $\Rightarrow$ **feuille « oui »**, on s'arrête.

Âge ≤ 30 (2 oui, 3 non, entropie 0,971). On recalcule le gain des attributs *restants* dans ce sous-ensemble de 5 individus :

- Étudiant : oui $\rightarrow$ {oui,oui} (pur), non $\rightarrow$ {non,non,non} (pur) $\Rightarrow E_{\text{reste}} = 0$, **gain = 0,971 (maximal)**;
- Revenu : gain = 0,571; État_civil : gain = 0,020.

$\Rightarrow$ on découpe sur **Étudiant** (sépare parfaitement).

Âge > 40 (3 oui, 2 non, entropie 0,971) :

- État_civil : célibataire $\rightarrow$ {3 oui} (pur), marié $\rightarrow$ {2 non} (pur) $\Rightarrow$ **gain = 0,971 (maximal)**;
- Étudiant : gain = 0,020; Revenu : gain = 0,020.

$\Rightarrow$ on découpe sur **État_civil** (sépare parfaitement).

Exemple — arbre gain d'information final

Âge ?

|-- (<= 30) > Étudiant ? oui > **oui** / non > **non**

|-- (31..40) > **oui**

|-- (> 40) > État_civil ? célibataire > **oui** / marié > **non**

Arbre de décision — Gain d'information / entropie (scikit-learn)

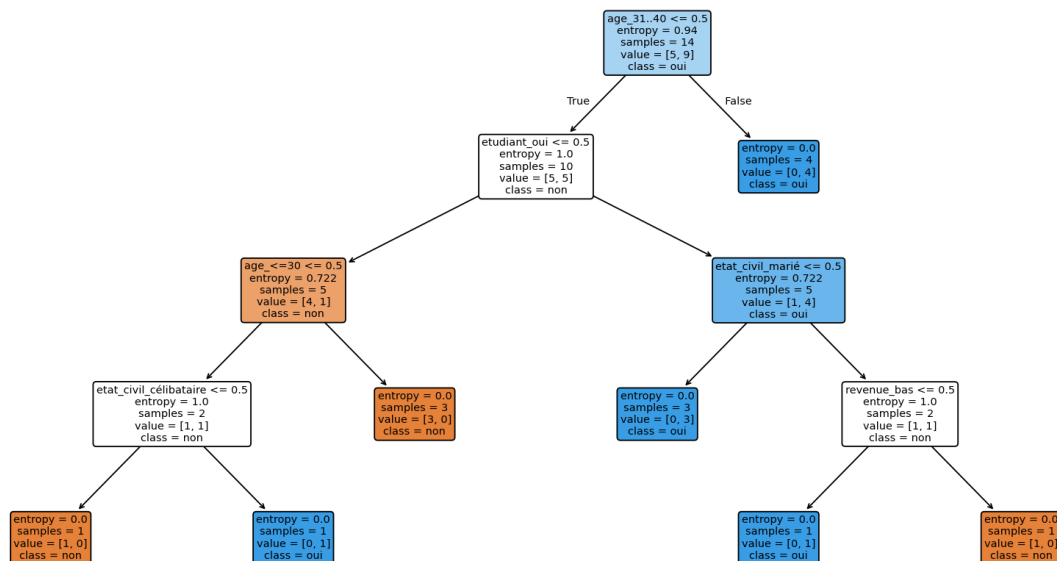


Fig. 3.3 : Arbre par gain d'information (entropie) reconstruit par scikit-learn.

Listing 3.3 : scripts/gain_information.py

```

1 """
2 CHAPITRE 4 / TP – ARBRE DE DÉCISION par le GAIN D'INFORMATION (ID3 / C4.5)
3 =====
4 Données (achat d'ordinateur) – 14 individus :
5   age{<=30, 31..40, >40}, revenue{élevé, moyen, bas}, étudiant{oui, non},
6   Etat_civil{célibataire, marié}, classe = Achète_ordinateur{oui, non}
7
8 RAPPELS (cours), logarithme en base 2 :
9   Entropie(t) = - somme_j p(j|t) * log2 p(j|t)      (0 = pur, 1 = 50/50)

```

```

10 Gain(A) = Entropie(parent) - somme_i (n_i/n) * Entropie(enfant_i)
11 -> on choisit l'attribut de PLUS GRAND gain (réduction d'entropie maximale).
12 """
13
14 import os
15 from math import log2
16
17 import matplotlib
18
19 matplotlib.use("Agg")
20 import matplotlib.pyplot as plt
21 import pandas as pd
22
23 FIG = os.path.join(os.path.dirname(__file__), "..", "rapport", "figures")
24 os.makedirs(FIG, exist_ok=True)
25
26 LIGNES = [
27     ("<=30", "élevé", "non", "célibataire", "non"),
28     ("<=30", "élevé", "non", "marié", "non"),
29     ("31..40", "élevé", "non", "célibataire", "oui"),
30     (">40", "moyen", "non", "célibataire", "oui"),
31     (">40", "bas", "oui", "célibataire", "oui"),
32     (">40", "bas", "oui", "marié", "non"),
33     ("31..40", "bas", "oui", "marié", "oui"),
34     ("<=30", "moyen", "non", "célibataire", "non"),
35     ("<=30", "bas", "oui", "célibataire", "oui"),
36     (">40", "moyen", "oui", "célibataire", "oui"),
37     ("<=30", "moyen", "oui", "marié", "oui"),
38     ("31..40", "moyen", "non", "marié", "oui"),
39     ("31..40", "élevé", "oui", "célibataire", "oui"),
40     (">40", "moyen", "non", "marié", "non"),
41 ]
42 COLS = ["age", "revenue", "etudiant", "etat_civil", "achete"]
43 DATA = pd.DataFrame(LIGNES, columns=COLS)
44 CIBLE = "achete"
45
46
47 def entropie(serie) -> float:
48     """- somme p*log2(p)."""
49     n = len(serie)
50     if n == 0:
51         return 0.0
52     h = 0.0
53     for c in serie.unique():
54         p = (serie == c).sum() / n
55         if p > 0:
56             h -= p * log2(p)
57     return h
58
59
60 def gain(df, attr) -> tuple[float, float]:
61     base = entropie(df[CIBLE])
62     reste = 0.0
63     for _, g in df.groupby(attr):
64         reste += len(g) / len(df) * entropie(g[CIBLE])
65     return base - reste, reste
66
67
68 def construire_id3(df, attributs, profondeur=0):
69     """Arbre ID3 récursif. Renvoie soit une classe (feuille) soit un dict."""
70     indent = " " * profondeur
71     classes = df[CIBLE].unique()
72     if len(classes) == 1: # ænud pur
73         print(f"{indent}-> feuille : {classes[0]}")

```

```

74     return classes[0]
75 if not attributs: # plus d'attribut : classe majoritaire
76     maj = df[CIBLE].mode()[0]
77     print(f"{indent}-> feuille (majorité) : {maj}")
78     return maj
79 # choisir l'attribut de plus grand gain
80 gains = {a: gain(df, a)[0] for a in attributs}
81 meilleur = max(gains, key=gains.get)
82 print(f"{indent}NUD : on teste << {meilleur} >> "
83       f"(gains: {{{', '.join(f'{a}:{g:.3f}' for a, g in gains.items())}}})")
84 arbre = {meilleur: {}}
85 for val, sous in df.groupby(meilleur):
86     print(f"{indent} {meilleur} = {val} :")
87     reste = [a for a in attributs if a != meilleur]
88     arbre[meilleur][val] = construire_id3(sous, reste, profondeur + 2)
89 return arbre
90
91
92 def main():
93     print("=" * 70)
94     print("ARBRE DE DÉCISION – GAIN D'INFORMATION (ID3)")
95     print("=" * 70)
96     print(DATA.to_string(index=False))
97
98     base = entropie(DATA[CIBLE])
99     no, yes = (DATA[CIBLE] == "non").sum(), (DATA[CIBLE] == "oui").sum()
100    print(f"\nEntropie(racine) = -({yes}/14)log2({yes}/14) - ({no}/14)log2({no}/14) "
101          f"= {base:.3f}")
102
103    print("\nGain de chaque attribut au 1er niveau :")
104    for a in ["age", "revenue", "etudiant", "etat_civil"]:
105        g, reste = gain(DATA, a)
106        print(f"    Gain({a:11s}) = {base:.3f} - {reste:.3f} = {g:.3f}")
107    print("    => 'age' a le plus grand gain : c'est la RACINE.\n")
108
109    print("CONSTRUCTION RÉCURSIVE :")
110    construire_id3(DATA, ["age", "revenue", "etudiant", "etat_civil"])
111
112    print("\nARBRE FINAL :")
113    print("    age ?")
114    print("    |<=30    -> étudiant ? oui->oui , non->non")
115    print("    | 31..40 -> oui    (feuille pure)")
116    print("    |>40     -> etat_civil ? célibataire->oui , marié->non")
117
118    # --- Vérification scikit-learn (entropy) + figure ---
119    from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
120
121    enc = pd.get_dummies(DATA.drop(columns=[CIBLE]))
122    y = DATA[CIBLE]
123    model = DecisionTreeClassifier(criterion="entropy", random_state=0).fit(enc, y)
124    print("\n[sklearn, criterion=entropy] règles :")
125    print(export_text(model, feature_names=list(enc.columns)))
126
127    plt.figure(figsize=(13, 7))
128    plot_tree(model, feature_names=list(enc.columns),
129              class_names=model.classes_, filled=True, rounded=True, fontsize=8)
130    plt.title("Arbre de décision – Gain d'information / entropie (scikit-learn)")
131    plt.tight_layout()
132    plt.savefig(os.path.join(FIG, "gain_information_arbre.png"), dpi=130)
133    print("[figure] rapport/figures/gain_information_arbre.png enregistrée")
134
135
136 if __name__ == "__main__":
137     main()

```

3.3 Mesures de performance

À partir de la **matrice de confusion** (cas binaire) :

	prédit positif	prédit négatif
réel positif	TP (vrais positifs)	FN (faux négatifs)
réel négatif	FP (faux positifs)	TN (vrais négatifs)

$$\text{Exactitude} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Précision} = \frac{TP}{TP + FP}, \quad \text{Rappel} = \frac{TP}{TP + FN}$$

$$\text{Spécificité} = \frac{TN}{TN + FP}, \quad F_1 = \frac{2 \cdot \text{Précision} \cdot \text{Rappel}}{\text{Précision} + \text{Rappel}}$$

Calcul détaillé : exemple TP=40, FN=10, FP=5, TN=45

Exactitude = $\frac{40+45}{100} = 0,85$; Précision = $\frac{40}{45} = 0,889$; Rappel = $\frac{40}{50} = 0,80$; Spécificité = $\frac{45}{50} = 0,90$; $F_1 = \frac{2 \cdot 0,889 \cdot 0,80}{0,889 + 0,80} = 0,842$.

Multi-classes : matrice $m \times m$, bonnes affectations $S = \sum_i n_{ii}$ (diagonale), taux d'erreur $E = 1 - \frac{\sum_i n_{ii}}{\sum_{i,j} n_{ij}}$.

Matrice de confusion

	prédit +	prédit -
réel +	40	10
réel -	5	45

Fig. 3.4 : Matrice de confusion de l'exemple.

Listing 3.4 : scripts/performance_classification.py

```

1  """
2  CHAPITRE 4 – MESURES DE PERFORMANCE D'UN CLASSIFIEUR
3  =====
4  À partir de la MATRICE DE CONFUSION (cas binaire) :
5      prédit positif  prédit négatif
6  réel positif      TP          FN
7  réel négatif      FP          TN
8
9  FORMULES (cours) :
10  Exactitude = (TP+TN) / (TP+TN+FP+FN)  -> taux de bonnes classifications
11  Précision  = TP / (TP+FP)              -> parmi les "positifs prédits", combien justes
12  Rappel     = TP / (TP+FN)              -> parmi les vrais positifs, combien retrouvés
13  Spécificité = TN / (TN+FP)             -> taux de vrais négatifs retrouvés
14  F1-score   = 2*P*R / (P+R)            -> moyenne harmonique précision/rappel
15  """
16
17  import os

```

```

18
19 import matplotlib
20
21 matplotlib.use("Agg")
22 import matplotlib.pyplot as plt
23
24 FIG = os.path.join(os.path.dirname(__file__), "..", "rapport", "figures")
25 os.makedirs(FIG, exist_ok=True)
26
27
28 def mesures(tp, fn, fp, tn):
29     exactitude = (tp + tn) / (tp + tn + fp + fn)
30     precision = tp / (tp + fp) if (tp + fp) else 0.0
31     rappel = tp / (tp + fn) if (tp + fn) else 0.0
32     specificite = tn / (tn + fp) if (tn + fp) else 0.0
33     f1 = 2 * precision * rappel / (precision + rappel) if (precision + rappel) else 0.0
34     return exactitude, precision, rappel, specificite, f1
35
36
37 def main():
38     print("=" * 70)
39     print("MESURES DE PERFORMANCE - exemple")
40     print("=" * 70)
41     # exemple : 100 individus, classe "positif" = malade
42     TP, FN, FP, TN = 40, 10, 5, 45
43     print(f"Matrice de confusion : TP={TP}, FN={FN}, FP={FP}, TN={TN}")
44
45     ex, pr, ra, sp, f1 = mesures(TP, FN, FP, TN)
46     print(f"\nExactitude = (TP+TN)/total = ({TP}+{TN})/{TP+FN+FP+TN} = {ex:.3f}")
47     print(f"Précision = TP/(TP+FP) = {TP}/({TP}+{FP}) = {pr:.3f}")
48     print(f"Rappel = TP/(TP+FN) = {TP}/({TP}+{FN}) = {ra:.3f}")
49     print(f"Spécificité = TN/(TN+FP) = {TN}/({TN}+{FP}) = {sp:.3f}")
50     print(f"F1-score = 2PR/(P+R) = {f1:.3f}")
51
52     # figure matrice de confusion
53     import numpy as np
54
55     M = np.array([[TP, FN], [FP, TN]])
56     fig, ax = plt.subplots(figsize=(4.5, 4))
57     ax.imshow(M, cmap="Blues")
58     for (i, j), v in np.ndenumerate(M):
59         ax.text(j, i, str(v), ha="center", va="center", fontsize=16,
60               color="white" if v > M.max() / 2 else "black")
61     ax.set_xticks([0, 1], ["prédit +", "prédit -"])
62     ax.set_yticks([0, 1], ["réel +", "réel -"])
63     ax.set_title("Matrice de confusion")
64     fig.tight_layout()
65     fig.savefig(os.path.join(FIG, "matrice_confusion.png"), dpi=130)
66     print("[figure] rapport/figures/matrice_confusion.png enregistrée")
67
68
69 if __name__ == "__main__":
70     main()

```

En résumé

K-NN : pas d'apprentissage, on vote parmi les K voisins les plus proches (normaliser avant !).
Arbres : on découpe récursivement selon le meilleur attribut — **Gini** (à minimiser, CART) ou **gain d'information** (à maximiser, ID3/C4.5). **Performance** : matrice de confusion → exactitude, précision, rappel, F_1 .

Segmentation (Clustering)

Définition Clustering

Organiser les données en **groupes (clusters)** de sorte que les données **similaires** soient ensemble. C'est de l'apprentissage **non supervisé** : les **classes sont inconnues**, on les découvre.

Astuce / À retenir : ce qu'est un « bon » clustering

Forte similarité intra-classe (les points d'un même groupe se ressemblent) **et faible similarité inter-classes** (les groupes sont bien séparés).

4.1 Mesurer la qualité d'un cluster

Chaque cluster C_k (N_k points) est caractérisé par :

$$\text{centre de gravité } \mu_k = \frac{1}{N_k} \sum_{x_i \in C_k} x_i, \quad \text{inertie } J_k = \sum_{x_i \in C_k} d^2(x_i, \mu_k).$$

Inertie intra (faible = points concentrés autour du centre); **inertie inter** (grande = centres éloignés, clusters bien séparés). **Objectif** : minimiser l'intra et maximiser l'inter.

4.2 Clustering hiérarchique

On construit un **dendrogramme** (arbre binaire de groupes). On le **coupe** horizontalement pour obtenir le nombre de groupes voulu.

- **Ascendant (agglomératif)** — le plus courant : on part de n singletons et on **fusionne** les groupes les plus proches jusqu'à un seul groupe.
- **Descendant (divisif)** : on part d'un groupe et on le divise.

Distance entre groupes : **lien simple** (min des distances), **lien complet** (max), **lien moyen** (moyenne).

4.2.1 Exercice résolu — lien simple

Matrice de distance :

	B	C	D	E
A	1	3	2	4
B		3	2	3
C			1	3
D				5

Calcul détaillé : fusions successives (lien simple = min)

Étape 1 : plus petite distance = 1 pour (A, B) et (C, D) . On fusionne $\{A, B\}$ (distance 1). Mise à jour (min) : $d(\{A, B\}, C) = \min(3, 3) = 3$, $d(\{A, B\}, D) = \min(2, 2) = 2$, $d(\{A, B\}, E) = \min(4, 3) = 3$.

Étape 2 : plus petite = $d(C, D) = 1 \rightarrow$ fusion $\{C, D\}$. Mise à jour : $d(\{A, B\}, \{C, D\}) =$

$\min(3, 2) = 2$, $d(\{C, D\}, E) = \min(3, 5) = 3$.

Étape 3 : plus petite = $d(\{A, B\}, \{C, D\}) = 2 \rightarrow$ fusion $\{A, B, C, D\}$. Reste $d(\{A, B, C, D\}, E) = \min(3, 3) = 3$.

Étape 4 : fusion finale avec E à la hauteur **3**.

Exemple — dendrogramme

Hauteurs de fusion : $\{A, B\}$ et $\{C, D\}$ à **1**, puis $\{AB, CD\}$ à **2**, puis $\{ABCD, E\}$ à **3**. En coupant à hauteur 2,5 on obtient **2 groupes** : $\{A, B, C, D\}$ et $\{E\}$.

Astuce / À retenir : et avec un autre lien ? (comparaison sur la même matrice)

Le *lien* change la façon de mesurer la distance entre groupes, donc les hauteurs de fusion. Sur la même matrice :

Lien	Fusions (groupe @ hauteur)
Simple (min)	$\{AB\}@1$, $\{CD\}@1$, $\{ABCD\}@2$, $\{ABCDE\}@3$
Complet (max)	$\{AB\}@1$, $\{CD\}@1$, $\{ABCD\}@3$, $\{ABCDE\}@5$

Détail du *lien complet* (on prend le *max*) : après $\{A, B\}$ et $\{C, D\}$, $d(\{A, B\}, \{C, D\}) = \max(3, 2, 3, 2) = 3$ (au lieu de 2), et la dernière fusion avec E se fait à $\max(4, 3, 3, 5) = 5$ (au lieu de 3). Le lien complet donne des clusters plus « compacts », le lien simple peut produire des « chaînes » allongées.

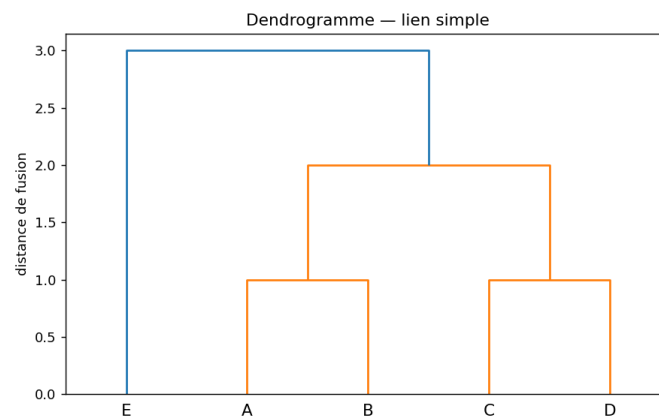


Fig. 4.1 : Dendrogramme (lien simple) obtenu avec SciPy — cohérent avec le calcul à la main.

Listing 4.1 : scripts/hierarchique_lien_simple.py

```

1  """
2  CHAPITRE 5 – CLUSTERING HIÉRARCHIQUE ASCENDANT (lien simple)
3  =====
4  EXERCICE : à partir de la matrice de distance ci-dessous, réaliser le
5  clustering hiérarchique ascendant avec le LIEN SIMPLE (single linkage).
6
7  Matrice de distance (symétrique) :
8      B  C  D  E
9  A  1  3  2  4
10 B      3  2  3
11 C          1  3
12 D              5
13

```

```

14 LIEN SIMPLE : distance entre 2 groupes = MIN des distances entre leurs points.
15 On fusionne à chaque étape les 2 groupes les plus proches, jusqu'à 1 seul groupe.
16 """
17
18 import os
19
20 import matplotlib
21
22 matplotlib.use("Agg")
23 import matplotlib.pyplot as plt
24 import numpy as np
25 from scipy.cluster.hierarchy import dendrogram, linkage
26
27 FIG = os.path.join(os.path.dirname(__file__), "..", "rapport", "figures")
28 os.makedirs(FIG, exist_ok=True)
29
30 POINTS = ["A", "B", "C", "D", "E"]
31 # distances initiales (paires)
32 D0 = {
33     ("A", "B"): 1, ("A", "C"): 3, ("A", "D"): 2, ("A", "E"): 4,
34     ("B", "C"): 3, ("B", "D"): 2, ("B", "E"): 3,
35     ("C", "D"): 1, ("C", "E"): 3,
36     ("D", "E"): 5,
37 }
38
39
40 def dist(a, b):
41     return D0[(a, b)] if (a, b) in D0 else D0[(b, a)]
42
43
44 def lien_simple(g1, g2):
45     """min des distances entre tout point de g1 et tout point de g2."""
46     return min(dist(x, y) for x in g1 for y in g2)
47
48
49 def main():
50     print("=" * 70)
51     print("CLUSTERING HIÉRARCHIQUE ASCENDANT – LIEN SIMPLE")
52     print("=" * 70)
53
54     # chaque point dans son propre cluster, représenté par un tuple de membres
55     clusters = [(p,) for p in POINTS]
56     etape = 0
57     while len(clusters) > 1:
58         etape += 1
59         # trouver la paire de clusters la plus proche (lien simple)
60         meilleure = (float("inf"), None, None)
61         for i in range(len(clusters)):
62             for j in range(i + 1, len(clusters)):
63                 d = lien_simple(clusters[i], clusters[j])
64                 if d < meilleure[0]:
65                     meilleure = (d, i, j)
66         d, i, j = meilleure
67         ci, cj = clusters[i], clusters[j]
68         fusion = ci + cj
69         print(f"\nÉtape {etape} : plus proches = {ci} et {cj} (distance lien simple = {d})")
70         print(f"        -> fusion en {fusion}")
71         clusters = [c for k, c in enumerate(clusters) if k not in (i, j)] + [fusion]
72         # afficher la matrice mise à jour (lien simple) entre clusters restants
73         if len(clusters) > 1:
74             print("    distances mises à jour :")
75             for a in range(len(clusters)):
76                 for b in range(a + 1, len(clusters)):
77                     print(f"        d({clusters[a]}, {clusters[b]}) = "

```

```

78         f"{lien_simple(clusters[a], clusters[b])}")
79
80     print(f"\nRésultat : un seul groupe {clusters[0]}")
81     print("Ordre des fusions (dendrogramme) : {A,B}@1, {C,D}@1, {AB,CD}@2, {ABCD,E}@3")
82
83     # --- dendrogramme avec scipy (vérification + figure) ---
84     # matrice condensée au format scipy : ordre AB, AC, AD, AE, BC, BD, BE, CD, CE, DE
85     condensee = np.array([
86         dist("A", "B"), dist("A", "C"), dist("A", "D"), dist("A", "E"),
87         dist("B", "C"), dist("B", "D"), dist("B", "E"),
88         dist("C", "D"), dist("C", "E"),
89         dist("D", "E"),
90     ], dtype=float)
91     Z = linkage(condensee, method="single")
92     print("\n[scipy] matrice de liaison (cluster1, cluster2, distance, taille) :")
93     print(Z)
94
95     plt.figure(figsize=(7, 4.5))
96     dendrogram(Z, labels=POINTS)
97     plt.title("Dendrogramme - lien simple")
98     plt.ylabel("distance de fusion")
99     plt.tight_layout()
100    plt.savefig(os.path.join(FIG, "hierarchique_dendrogramme.png"), dpi=130)
101    print("[figure] rapport/figures/hierarchique_dendrogramme.png enregistrée")
102
103
104 if __name__ == "__main__":
105     main()

```

4.3 Clustering par partitionnement — K-means

Définition K-means

On fixe K à l'avance. On alterne deux étapes jusqu'à convergence : **(1) Affectation** — chaque point va au centre μ_ℓ le plus proche ; **(2) Mise à jour** — chaque centre devient la *moyenne* de son cluster. Critère d'arrêt : les centres ne bougent plus, $\sum_i \|\mu_i^t - \mu_i^{t-1}\|^2 \leq \epsilon$.

4.3.1 Exercice résolu — $C = \{2, 3, 4, 10, 11, 12, 20, 25, 30\}$, $K = 2$, $\mu_1 = 2, \mu_2 = 4$

À chaque itération : pour chaque point on compare $|x - \mu_1|$ et $|x - \mu_2|$, on l'affecte au plus petit ; puis on recalcule chaque centre comme *moyenne* de son groupe.

Calcul détaillé : itération 1 en entier ($\mu_1=2, \mu_2=4$)

x	$ x - 2 $	$ x - 4 $	le plus proche	groupe
2	0	2	μ_1	C_1
3	1	1	égalité $\rightarrow \mu_1$	C_1
4	2	0	μ_2	C_2
10,11,12,20,25,30	grand	moins grand	μ_2	C_2

$$C_1 = \{2, 3\} \rightarrow \mu_1 = \frac{2+3}{2} = 2,5; \quad C_2 = \{4, 10, 11, 12, 20, 25, 30\} \rightarrow \mu_2 = \frac{112}{7} = 16.$$

Calcul détaillé : itérations 2 à 5 (résumé des recalculs)

It. 2 (2,5; 16) : $C_1 = \{2, 3, 4\} \rightarrow \mu_1 = 3$; $C_2 = \{10, \dots, 30\} \rightarrow \mu_2 = 18$.

It. 3 (3; 18) : 10 bascule en C_1 car $|10 - 3| = 7 < |10 - 18| = 8$. $C_1 = \{2, 3, 4, 10\} \rightarrow \mu_1 = 4,75$;

$C_2 = \{11, 12, 20, 25, 30\} \rightarrow \mu_2 = 19,6$.

It. 4 (4,75; 19,6) : 11 et 12 basculent en C_1 ($|12 - 4,75| = 7,25 < |12 - 19,6| = 7,6$). $C_1 = \{2, 3, 4, 10, 11, 12\} \rightarrow \mu_1 = 7$; $C_2 = \{20, 25, 30\} \rightarrow \mu_2 = 25$.

It. 5 (7; 25) : aucune réaffectation, mêmes moyennes $\Rightarrow$ **convergence** ($\sum_i \|\mu_i^t - \mu_i^{t-1}\|^2 = 0 \leq \epsilon$).

Exemple — résultat

$C_1 = \{2, 3, 4, 10, 11, 12\}$ (centre 7) $C_2 = \{20, 25, 30\}$ (centre 25)

Calcul détaillé : qualité finale — l'inertie (intra-cluster)

$J_k = \sum_{x \in C_k} (x - \mu_k)^2$ mesure la dispersion d'un cluster autour de son centre.

$$J_1 = (2-7)^2 + (3-7)^2 + (4-7)^2 + (10-7)^2 + (11-7)^2 + (12-7)^2 = 25+16+9+9+16+25 = 100$$

$$J_2 = (20-25)^2 + (25-25)^2 + (30-25)^2 = 25+0+25 = 50$$

Inertie totale $J = J_1 + J_2 = 150$. C'est exactement le *within-cluster sum of squares* affiché par scikit-learn (`inertia_`) et par WEKA. Plus elle est faible, plus les clusters sont « serrés ».

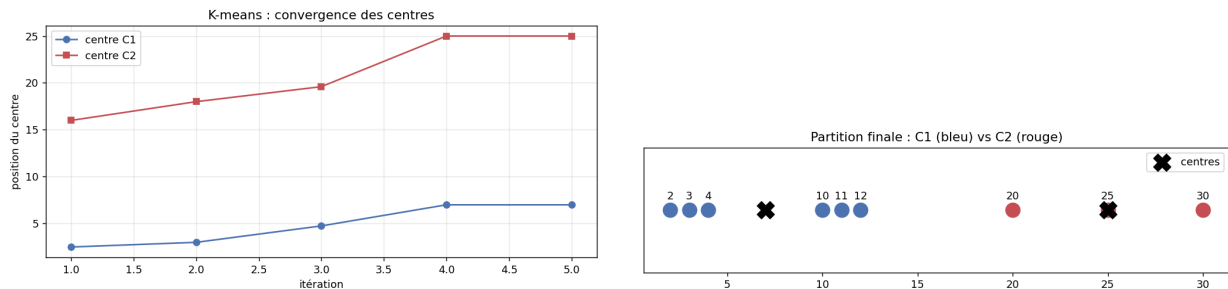


Fig. 4.2 : À gauche : déplacement des centres au fil des itérations. À droite : partition finale.

Attention

K-means dépend de l'**initialisation** des centres et exige des **distances** (donc **normaliser** les attributs). Il faut aussi **choisir** K (méthode du coude, silhouette...).

Listing 4.2 : scripts/kmeans_1d.py

```
1 """
2 CHAPITRE 5 - SEGMENTATION : algorithme K-MEANS (centres mobiles)
3 =====
4 EXERCICE : C = {2, 3, 4, 10, 11, 12, 20, 25, 30}
5           k = 2, centres initiaux mu1 = 2 , mu2 = 4
6
7 PRINCIPE (cours) :
8   1. AFFECTATION : chaque point va au centre le plus proche.
9   2. MISE À JOUR : chaque centre devient la moyenne des points de son cluster.
10  On répète jusqu'à ce que les centres ne bougent plus (convergence).
11 """
12
13 import os
14
15 import matplotlib
16
```

```

17 matplotlib.use("Agg")
18 import matplotlib.pyplot as plt
19
20 FIG = os.path.join(os.path.dirname(__file__), "..", "rapport", "figures")
21 os.makedirs(FIG, exist_ok=True)
22
23 C = [2, 3, 4, 10, 11, 12, 20, 25, 30]
24
25
26 def kmeans_1d(points, mu1, mu2, max_iter=20):
27     historique = []
28     for it in range(1, max_iter + 1):
29         # 1) AFFECTATION : point vers le centre le plus proche (|.|)
30         g1, g2 = [], []
31         for x in points:
32             (g1 if abs(x - mu1) <= abs(x - mu2) else g2).append(x)
33         # 2) MISE À JOUR : moyenne de chaque groupe
34         n1 = sum(g1) / len(g1) if g1 else mu1
35         n2 = sum(g2) / len(g2) if g2 else mu2
36         historique.append((it, mu1, mu2, list(g1), list(g2), n1, n2))
37         if n1 == mu1 and n2 == mu2: # convergence
38             break
39         mu1, mu2 = n1, n2
40     return historique
41
42
43 def main():
44     print("=" * 70)
45     print("K-MEANS - C = {2,3,4,10,11,12,20,25,30}, k=2, mu1=2, mu2=4")
46     print("=" * 70)
47     hist = kmeans_1d(C, 2.0, 4.0)
48
49     for it, m1, m2, g1, g2, n1, n2 in hist:
50         print(f"\nItération {it} : centres mu1={m1}, mu2={m2}")
51         for x in C:
52             d1, d2 = abs(x - m1), abs(x - m2)
53             choix = "C1" if d1 <= d2 else "C2"
54             print(f"    x={x:2d} : |{x}-{m1}|={d1:5.2f} |{x}-{m2}|={d2:5.2f} -> {choix}")
55         print(f"    C1 = {g1} -> nouvelle moyenne = {n1:.3f}")
56         print(f"    C2 = {g2} -> nouvelle moyenne = {n2:.3f}")
57
58     it, m1, m2, g1, g2, n1, n2 = hist[-1]
59     print("\nCONVERGENCE :")
60     print(f"    Cluster 1 = {g1} (centre = {n1})")
61     print(f"    Cluster 2 = {g2} (centre = {n2})")
62
63     # inertie intra-cluster = somme des carrés des écarts au centre
64     J1 = sum((x - n1) ** 2 for x in g1)
65     J2 = sum((x - n2) ** 2 for x in g2)
66     print(f"    Inertie J1 = {J1}, J2 = {J2}, total J = {J1 + J2}")
67
68     # Vérification scikit-learn
69     try:
70         import numpy as np
71         from sklearn.cluster import KMeans
72
73         km = KMeans(n_clusters=2, init=np.array([[2.0], [4.0]]), n_init=1).fit(
74             np.array(C).reshape(-1, 1)
75         )
76         print(f"\n[sklearn] centres = {sorted(km.cluster_centers_.ravel().round(3))}")
77     except Exception as e: # pragma: no cover
78         print("sklearn indisponible:", e)
79
80     # Figure de la convergence (centres au fil des itérations)

```

```

81 fig, ax = plt.subplots(figsize=(8, 4))
82 its = [h[0] for h in hist]
83 ax.plot(its, [h[5] for h in hist], "o-", label="centre C1", color="#4C72B0")
84 ax.plot(its, [h[6] for h in hist], "s-", label="centre C2", color="#C44E52")
85 ax.set_xlabel("itération")
86 ax.set_ylabel("position du centre")
87 ax.set_title("K-means : convergence des centres")
88 ax.legend()
89 ax.grid(alpha=0.3)
90 fig.tight_layout()
91 fig.savefig(os.path.join(FIG, "kmeans_convergence.png"), dpi=130)
92
93 # Figure des points colorés par cluster final
94 fig2, ax2 = plt.subplots(figsize=(8, 2.4))
95 for x in C:
96     col = "#4C72B0" if x in g1 else "#C44E52"
97     ax2.scatter(x, 0, s=180, color=col, zorder=3)
98     ax2.annotate(str(x), (x, 0), textcoords="offset points", xytext=(0, 10), ha="center")
99 ax2.scatter([n1, n2], [0, 0], marker="X", s=260, color="black", zorder=4, label="centres")
100 ax2.set_yticks([])
101 ax2.set_title("Partition finale : C1 (bleu) vs C2 (rouge)")
102 ax2.legend()
103 fig2.tight_layout()
104 fig2.savefig(os.path.join(FIG, "kmeans_partition.png"), dpi=130)
105 print("[figures] kmeans_convergence.png et kmeans_partition.png enregistrées")
106
107
108 if __name__ == "__main__":
109     main()

```

En résumé

Hierarchique : fusions successives (lien simple/complet/moyen) → dendrogramme que l'on coupe. **K-means** : K fixé, on alterne affectation ↔ recalcul des moyennes jusqu'à stabilité. Les deux cherchent **forte cohésion interne** et **forte séparation** entre groupes.

TP « CLASSIFICATION » — corrigé complet

Ce TP (document scanné) demande de **dessiner deux arbres de décision** et de **vérifier avec WEKA et PYTHON**. Les deux exercices reprennent exactement les jeux de données traités au chapitre 4 ; on en redonne ici la résolution condensée et la procédure de vérification.

5.1 Exercice 1 — Indice de Gini

ID	Âge	Type_voit	Risque
0	23	Familiale	Élevé
1	17	Sport	Élevé
2	43	Sport	Élevé
3	68	Familiale	Faible
4	32	Camion	Faible
5	20	Familiale	Élevé

Résolution (détaillée au §4.2.1) : $GINI(\text{racine}) = 0,444$. Les deux meilleurs premiers découpages (Type_voit et Âge < 27,5) sont à égalité ($GINI_{\text{split}} = 0,222$) ; scikit-learn retient **Âge** < 27,5, puis **Type_voit** sépare parfaitement.

Exemple — arbre attendu

Âge < 27,5 ? → OUI : **Élevé** ; NON : Type_voit → Sport=**Élevé**, Familiale=**Faible**, Camion=**Faible**.

Script : `uv run python scripts/gini_arbre.py` (figure gini_arbre.png).

5.2 Exercice 2 — Gain d'information

Jeu « achat d'ordinateur » (14 lignes, §4.2.2) : $Entropie(S) = 0,940$. Gains : Âge = 0,246 (max), Étudiant = 0,151, État_civil = 0,048, Revenu = 0,029.

Exemple — arbre attendu

Âge ? → ≤ 30 : Étudiant (oui=oui, non=non) ; 31..40 : **oui** ; > 40 : État_civil (célibataire=oui, marié=non).

Script : `uv run python scripts/gain_information.py` (figure gain_information_arbre.png).

5.3 Vérification sous WEKA

1. Enregistrer le tableau au format `.arff` ou `.csv`.
2. *Explorer* → onglet *Classify*.
3. Choisir `trees > J48` (implémentation de C4.5, $\approx$ gain d'information) ou `trees > SimpleCart` (indice de Gini).
4. *Test options* : « Use training set » (jeux minuscules) puis lancer.

5. Lire l'arbre dans la sortie texte et le comparer à celui de Python — **ils coïncident** (même attribut racine, mêmes feuilles).

Astuce / À retenir

WEKA J48 utilise le *gain ratio* (C4.5) : sur ces petits jeux, le résultat est identique à l'arbre par gain d'information ci-dessus. Pour Gini, utiliser SimpleCart/CART.

En résumé

Les deux arbres du TP sont exactement ceux du chapitre 4. Python (scikit-learn) et WEKA (J48 / SimpleCart) reconstruisent les mêmes structures, ce qui valide la résolution manuelle.

Le projet de Data Mining — application Streamlit

6.1 Le sujet

Définition Cahier des charges

Concevoir une **application graphique** (Java ou Python) exécutant les **3 tâches** de Data Mining : **segmentation**, **classification**, **recherche d'associations**. Fonctionnalités exigées :

1. choisir la tâche ;
2. renseigner le chemin du dataset ;
3. choisir l'algorithme ;
4. lancer l'analyse ;
5. afficher les résultats ;
6. un algorithme au choix par tâche ;
7. présenter les **mesures de performance** ;
8. **comparer avec WEKA**.

Datasets suggérés : *cereals* (segmentation), *customer churn / ionosphere* (classification), *customer churn* (associations).

6.2 Architecture de la solution

On a choisi **Python + Streamlit** (interface web instantanée, idéale pour un *dashboard* de data mining). Deux fichiers :

- `projet/apriori.py` — l'algorithme **Apriori** codé à la main (support, confiance, lift) ;
- `projet/app.py` — l'application : barre latérale (choix tâche + source des données), puis une page par tâche.

Astuce / À retenir : lancer l'application

```
uv run streamlit run projet/app.py
```

Sans fichier, l'app utilise des **jeux de démonstration** intégrés (type *cereals*, *churn*, paniers de supermarché). On peut aussi **importer un CSV**.

6.3 Les 3 tâches dans l'application

1) Segmentation. Algorithmes **K-means** et **Agglomératif** (hiérarchique). L'utilisateur choisit les variables, K , et la normalisation (z-score). Sorties : **silhouette** et **inertie intra-cluster** (mesures de performance), projection 2D (PCA) colorée par cluster, tailles des clusters, export CSV. **WEKA** : Cluster $\rightarrow$ SimpleKMeans, comparer le *within-cluster sum of squared errors* (= inertie).

2) Classification. Algorithmes **Arbre de décision**, **K-NN** (avec standardisation automatique), **Forêt aléatoire**. Découpage apprentissage/test paramétrable. Sorties : **exactitude**, **pré-**

cision, rappel, F1 (§4.4), **matrice de confusion**, tracé de l'arbre ou importance des variables.
WEKA : `Classify` → `J48` / `IBk` / `RandomForest`, comparer l'accuracy.

3) Recherche d'associations. Algorithme **Apriori** (maison). Deux formats : colonne « panier » (articles séparés par des virgules) ou tableau catégoriel (**colonne=valeur**). Réglages : **support** et **confiance** minimaux. Sorties : table des règles $X \rightarrow Y$ avec **support**, **confiance**, **lift**, top règles, export CSV. **WEKA** : `Associate` → `Apriori`, mêmes seuils.

6.4 Mesures de performance & comparaison WEKA

Tâche	Mesures dans l'app	Équivalent WEKA
Segmentation	Silhouette, inertie intra	SimpleKMeans (SSE)
Classification	Exactitude, précision, rappel, F1, matrice de confusion	J48 / IBk / RandomForest
Associations	Support, confiance, lift	Apriori (lift)

6.5 Code — algorithme Apriori (maison)

Listing 6.1 : projet/apriori.py

```

1  """
2  RÈGLES D'ASSOCIATION – algorithme APRIORI (implémenté à la main)
3  =====
4  Vocabulaire :
5  - transaction : un panier (ensemble d'articles), ex. {pain, lait, vin}
6  - itemset      : un ensemble d'articles
7  - support(X)   = proportion de transactions contenant X
8  - règle X -> Y : "si X alors Y"
9      confiance(X->Y) = support(X ∪ Y) / support(X)
10     lift(X->Y)      = confiance / support(Y)    (>1 = corrélation positive)
11
12  APRIORI repose sur une idée simple : si un itemset est fréquent, tous ses
13  sous-ensembles le sont aussi. Donc on construit les itemsets fréquents de
14  taille croissante, en n'élargissant que ceux qui ont passé le seuil de support.
15  """
16
17  from itertools import combinations
18
19
20  def _support(itemset, transactions):
21      """proportion de transactions qui contiennent tous les articles de itemset."""
22      s = set(itemset)
23      n = sum(1 for t in transactions if s.issubset(t))
24      return n / len(transactions)
25
26
27  def itemsets_frequents(transactions, min_support):
28      """Renvoie {frozenset: support} pour tous les itemsets fréquents."""
29      transactions = [set(t) for t in transactions]
30      # niveau 1 : articles individuels fréquents
31      articles = sorted({a for t in transactions for a in t})
32      courant = []
33      frequents = {}
34      for a in articles:
35          sup = _support([a], transactions)
36          if sup >= min_support:
37              courant.append((a,))

```

```

38         frequents[frozenset([a])] = sup
39
40     k = 2
41     while courant:
42         # génération des candidats de taille k par union des fréquents précédents
43         candidats = set()
44         items = sorted({a for c in courant for a in c})
45         for combo in combinations(items, k):
46             # élagage Apriori : tous les sous-ensembles (k-1) doivent être fréquents
47             if all(frozenset(sub) in frequents for sub in combinations(combo, k - 1)):
48                 candidats.add(combo)
49         # filtrage par support
50         courant = []
51         for cand in sorted(candidats):
52             sup = _support(cand, transactions)
53             if sup >= min_support:
54                 courant.append(cand)
55                 frequents[frozenset(cand)] = sup
56         k += 1
57     return frequents
58
59
60 def regles_association(transactions, min_support=0.3, min_confidence=0.6):
61     """Génère les règles X -> Y dépassant les seuils de support et de confiance."""
62     transactions = [set(t) for t in transactions]
63     frequents = itemsets_frequents(transactions, min_support)
64     regles = []
65     for itemset, sup in frequents.items():
66         if len(itemset) < 2:
67             continue
68         items = list(itemset)
69         # toutes les façons de couper itemset en antécédent X et conséquent Y
70         for r in range(1, len(items)):
71             for X in combinations(items, r):
72                 X = frozenset(X)
73                 Y = itemset - X
74                 conf = sup / frequents[X]
75                 if conf >= min_confidence:
76                     lift = conf / frequents[Y]
77                     regles.append(
78                         {
79                             "antecedent": set(X),
80                             "consequent": set(Y),
81                             "support": round(sup, 3),
82                             "confidence": round(conf, 3),
83                             "lift": round(lift, 3),
84                         }
85                     )
86     return sorted(regles, key=lambda r: (r["confidence"], r["lift"]), reverse=True)
87
88
89 if __name__ == "__main__":
90     # petit exemple "panier de la ménagère"
91     paniers = [
92         {"pain", "lait", "vin"},
93         {"pain", "lait"},
94         {"vin", "fromage"},
95         {"pain", "vin", "fromage"},
96         {"pain", "lait", "vin", "fromage"},
97     ]
98     print("Itemsets fréquents (min_support=0.4) :")
99     for it, sup in itemsets_frequents(paniers, 0.4).items():
100         print(f" {set(it)} -> support={sup:.2f}")
101     print("\nRègles (min_conf=0.6) :")

```

```

102     for r in regles_association(paniers, 0.4, 0.6):
103         print(f" {r['antecedent']} -> {r['consequent']} "
104               f"(supp={r['support']}, conf={r['confidence']}, lift={r['lift']})")

```

6.6 Code — application Streamlit

Listing 6.2 : projet/app.py

```

1  """
2  PROJET DE DATA MINING – Application graphique (Streamlit)
3  =====
4  Réalise les 3 tâches demandées par le sujet :
5      1) SEGMENTATION          (K-means, Agglomératif)
6      2) CLASSIFICATION        (Arbre de décision, K-NN, Forêt aléatoire)
7      3) RECHERCHE D'ASSOCIATIONS (Apriori)
8
9  Fonctionnalités (cahier des charges du projet) :
10     - choisir la tâche
11     - renseigner le dataset (upload CSV) OU utiliser un jeu de démonstration
12     - choisir l'algorithme et ses paramètres
13     - lancer l'analyse
14     - afficher les résultats + mesures de performance
15     - comparaison indicative avec WEKA
16
17  Lancement : uv run streamlit run projet/app.py
18  """
19
20  import io
21
22  import matplotlib
23  import numpy as np
24  import pandas as pd
25  import streamlit as st
26
27  matplotlib.use("Agg")
28  import matplotlib.pyplot as plt
29  from sklearn.cluster import AgglomerativeClustering, KMeans
30  from sklearn.decomposition import PCA
31  from sklearn.ensemble import RandomForestClassifier
32  from sklearn.metrics import (
33      accuracy_score,
34      confusion_matrix,
35      f1_score,
36      precision_score,
37      recall_score,
38      silhouette_score,
39  )
40  from sklearn.model_selection import train_test_split
41  from sklearn.neighbors import KNeighborsClassifier
42  from sklearn.preprocessing import StandardScaler
43  from sklearn.tree import DecisionTreeClassifier, plot_tree
44
45  from apriori import regles_association
46
47  # -----
48  # CONFIG & STYLE
49  # -----
50  st.set_page_config(
51      page_title="Data Mining Studio",
52      page_icon=":material/database:",
53      layout="centered", # colonne centrée et plus étroite (moins "wide")
54  )
55
56  st.markdown(

```

```

57     """
58     <style>
59     /* titre centré avec dégradé sobre */
60     .big-title {font-size:2.2rem;font-weight:800;text-align:center;margin-bottom:0;
61         background:linear-gradient(90deg,#4C72B0,#6C3483);
62         -webkit-background-clip:text;-webkit-text-fill-color:transparent;}
63     .subtitle {text-align:center;color:#7A7A7A;margin-top:0;}
64     /* on resserre encore un peu la colonne centrale */
65     .block-container{max-width:880px;padding-top:2.2rem;}
66     /* cartes de métriques : fond clair + texte foncé, lisible quel que soit le thème */
67     [data-testid="stMetric"]{
68         background:#EEF2FA;border:1px solid #C9D6F0;border-radius:12px;padding:10px 14px;
69         text-align:center;}
70     [data-testid="stMetric"] *{color:#1F2A44 !important;}
71     [data-testid="stMetricValue"]{font-weight:800;}
72     [data-testid="stMetricLabel"] p{color:#3A4A6B !important;}
73     [data-testid="stMetricLabel"]{justify-content:center;}
74     </style>
75     """
76     unsafe_allow_html=True,
77 )
78
79
80 # -----
81 # JEUX DE DÉMONSTRATION (pour que l'app marche sans fichier)
82 # -----
83 @st.cache_data
84 def demo_segmentation():
85     """Type 'cereals' : caractéristiques nutritionnelles."""
86     rng = np.random.default_rng(0)
87     g1 = rng.normal([110, 1, 8], [10, 0.5, 1.5], (25, 3)) # céréales légères
88     g2 = rng.normal([150, 5, 2], [12, 1.0, 1.0], (25, 3)) # riches/sucrées
89     g3 = rng.normal([90, 3, 14], [8, 0.8, 2.0], (25, 3)) # fibres
90     X = np.vstack([g1, g2, g3])
91     return pd.DataFrame(X, columns=["calories", "protéines", "fibres"]).round(1)
92
93
94 @st.cache_data
95 def demo_classification():
96     """Type 'customer churn' : le client résilie-t-il ?"""
97     rng = np.random.default_rng(1)
98     n = 300
99     anciennete = rng.integers(1, 72, n)
100    minutes = rng.normal(180, 60, n).clip(0)
101    appels_service = rng.poisson(1.5, n)
102    forfait_intl = rng.integers(0, 2, n)
103    # règle "cachée" : churn si peu d'ancienneté + bcp d'appels au service client
104    proba = 1 / (1 + np.exp(-(0.06 * appels_service * 2 + 0.02 * (12 - anciennete) - 0.5)))
105    churn = (rng.random(n) < proba).astype(int)
106    return pd.DataFrame(
107        {
108            "anciennete": anciennete,
109            "minutes_jour": minutes.round(1),
110            "appels_service": appels_service,
111            "forfait_intl": forfait_intl,
112            "churn": np.where(churn == 1, "oui", "non"),
113        }
114    )
115
116
117 @st.cache_data
118 def demo_association():
119     """Type 'panier' : transactions de supermarché (1 ligne = 1 panier)."""
120     paniers = [

```

```

121     "pain,lait,vin", "pain,lait", "vin,fromage", "pain,vin,fromage",
122     "pain,lait,vin,fromage", "lait,fromage", "pain,vin", "vin,fromage,oeufs",
123     "pain,lait,oeufs", "vin,fromage", "pain,vin,fromage,oeufs", "lait,oeufs",
124     "pain,lait,vin", "vin,fromage", "pain,lait,fromage",
125 ]
126 return pd.DataFrame({"panier": paniers})
127
128
129 def telecharger(df, nom):
130     buf = io.StringIO()
131     df.to_csv(buf, index=False)
132     st.download_button("Télécharger les résultats (CSV)", buf.getvalue(),
133                       file_name=nom, mime="text/csv", icon=":material/download:")
134
135
136 # -----
137 # SIDEBAR – choix de la tâche et des données
138 # Icônes natives "Material Symbols" via la syntaxe :material/nom:
139 # -----
140 # libellés des tâches associés à leur icône Material
141 TACHES = {
142     "Segmentation": ":material/bubble_chart:",
143     "Classification": ":material/account_tree:",
144     "Recherche d'associations": ":material/shopping_cart:",
145 }
146
147 st.sidebar.markdown("### :material/database: Data Mining Studio")
148 tache = st.sidebar.radio(
149     ":material/checklist: Choisir la tâche",
150     list(TACHES),
151     format_func=lambda t: f"{TACHES[t]} {t}", # affiche l'icône devant le libellé
152 )
153
154 st.sidebar.divider()
155 source = st.sidebar.radio(":material/folder_open: Source des données",
156                          ["Jeu de démonstration", "Importer un CSV"])
157
158 fichier = None
159 if source == "Importer un CSV":
160     fichier = st.sidebar.file_uploader("Fichier .csv", type=["csv"])
161     sep = st.sidebar.selectbox("Séparateur", [",", ";", "\\t"], index=0)
162
163
164 def charger(demo_func):
165     if source == "Importer un CSV" and fichier is not None:
166         s = "\\t" if sep == "\\t" else sep
167         return pd.read_csv(fichier, sep=s)
168     return demo_func()
169
170
171 st.markdown('<p class="big-title">Data Mining Studio</p>', unsafe_allow_html=True)
172 st.markdown('<p class="subtitle">Master 2 IG/ID, ENEAM/UAC · segmentation · '
173             'classification · règles d'association</p>', unsafe_allow_html=True)
174 st.divider()
175
176
177 # =====
178 # 1) SEGMENTATION
179 # =====
180 if tache == "Segmentation":
181     st.header(":material/bubble_chart: Segmentation (clustering)")
182     df = charger(demo_segmentation)
183     st.dataframe(df.head(), width="stretch")
184

```

```

185 num_cols = df.select_dtypes("number").columns.tolist()
186 cols = st.multiselect("Variables numériques à utiliser", num_cols, default=num_cols)
187 c1, c2, c3 = st.columns(3)
188 algo = c1.selectbox("Algorithme", ["K-means", "Agglomératif (hiérarchique)"])
189 k = c2.slider("Nombre de clusters K", 2, 8, 3)
190 normaliser = c3.checkbox("Normaliser (z-score)", value=True)
191
192 if st.button("Lancer la segmentation", type="primary",
193             icon="material/play_arrow:", width="stretch") and cols:
194     X = df[cols].to_numpy()
195     if normaliser:
196         X = StandardScaler().fit_transform(X)
197
198     if algo == "K-means":
199         model = KMeans(n_clusters=k, n_init=10, random_state=0)
200     else:
201         model = AgglomerativeClustering(n_clusters=k)
202     labels = model.fit_predict(X)
203
204     # --- mesures de performance ---
205     sil = silhouette_score(X, labels) if k > 1 else float("nan")
206     m1, m2 = st.columns(2)
207     m1.metric("Silhouette → (11, +haut = mieux)", f"{sil:.3f}")
208     if algo == "K-means":
209         m2.metric("Inertie intra-cluster", f"{model.inertia_:.1f}")
210     else:
211         m2.metric("Clusters formés", k)
212
213     # --- projection 2D (PCA) ---
214     proj = PCA(n_components=2).fit_transform(X) if X.shape[1] > 2 else X[:, :2]
215     fig, ax = plt.subplots(figsize=(7, 5))
216     sc = ax.scatter(proj[:, 0], proj[:, 1], c=labels, cmap="tab10", s=40)
217     if algo == "K-means":
218         cen = PCA(n_components=2).fit(X).transform(model.cluster_centers_) \
219             if X.shape[1] > 2 else model.cluster_centers_[ :, :2]
220         ax.scatter(cen[:, 0], cen[:, 1], marker="X", s=250, c="black", label="centres")
221         ax.legend()
222     ax.set_title(f"{algo} - {k} clusters (projection 2D)")
223     st.pyplot(fig)
224
225     res = df.copy()
226     res["cluster"] = labels
227     st.subheader("Taille des clusters")
228     st.bar_chart(res["cluster"].value_counts().sort_index())
229     st.dataframe(res, width="stretch")
230     telecharger(res, "segmentation_resultats.csv")
231
232     st.info("**Comparaison WEKA** : *Cluster → SimpleKMeans* (mêmes K et "
233           "normalisation). WEKA affiche le `Within cluster sum of squared errors` "
234           "(= inertie) ; il doit être du même ordre de grandeur.",
235           icon="material/science:")
236
237     st.info("**Astuce du cours** : *normaliser avant un clustering (les distances "
238           "dépendent des échelles). Choisir K via la **silhouette** ou la méthode du "
239           "**coude**", icon="material/lightbulb:")
240
241
242 # =====
243 # 2) CLASSIFICATION
244 # =====
245 elif tache == "Classification":
246     st.header("material/account_tree: Classification (supervisée)")
247     df = charger(demo_classification)
248     st.dataframe(df.head(), width="stretch")

```

```

249
250 cible = st.selectbox("Variable cible (la classe à prédire)", df.columns,
251                      index=len(df.columns) - 1)
252 feats = [c for c in df.columns if c != cible]
253 c1, c2, c3 = st.columns(3)
254 algo = c1.selectbox("Algorithme",
255                    ["Arbre de décision", "K plus proches voisins", "Forêt aléatoire"])
256 test_size = c2.slider("Part du jeu de test (%)", 10, 50, 33) / 100
257 if algo == "Arbre de décision":
258     param = c3.slider("Profondeur max", 1, 10, 4)
259 elif algo == "K plus proches voisins":
260     param = c3.slider("K (voisins)", 1, 15, 5)
261 else:
262     param = c3.slider("Nombre d'arbres", 10, 200, 100, step=10)
263
264 if st.button("Entraîner & évaluer", type="primary",
265            icon="material/play_arrow:", width="stretch"):
266     X = pd.get_dummies(df[feats]) # encodage des variables nominales
267     y = df[cible].astype(str)
268     Xtr, Xte, ytr, yte = train_test_split(
269         X, y, test_size=test_size, random_state=0, stratify=y
270     )
271
272     if algo == "Arbre de décision":
273         model = DecisionTreeClassifier(max_depth=param, random_state=0)
274     elif algo == "K plus proches voisins":
275         # K-NN : on standardise (distances !)
276         sc = StandardScaler().fit(Xtr)
277         Xtr, Xte = sc.transform(Xtr), sc.transform(Xte)
278         model = KNeighborsClassifier(n_neighbors=param)
279     else:
280         model = RandomForestClassifier(n_estimators=param, random_state=0)
281     model.fit(Xtr, ytr)
282     yp = model.predict(Xte)
283
284     # --- mesures de performance ---
285     classes = sorted(y.unique())
286     avg = "binary" if len(classes) == 2 else "macro"
287     pos = classes[-1] if len(classes) == 2 else None
288     acc = accuracy_score(yte, yp)
289     prec = precision_score(yte, yp, average=avg, pos_label=pos, zero_division=0)
290     rec = recall_score(yte, yp, average=avg, pos_label=pos, zero_division=0)
291     f1 = f1_score(yte, yp, average=avg, pos_label=pos, zero_division=0)
292
293     a, b, c, d = st.columns(4)
294     a.metric("Exactitude", f"{acc:.1%}")
295     b.metric("Précision", f"{prec:.1%}")
296     c.metric("Rappel", f"{rec:.1%}")
297     d.metric("F1-score", f"{f1:.1%}")
298
299     # --- matrice de confusion ---
300     cm = confusion_matrix(yte, yp, labels=classes)
301     fig, ax = plt.subplots(figsize=(4.5, 4))
302     ax.imshow(cm, cmap="Blues")
303     for (i, j), v in np.ndenumerate(cm):
304         ax.text(j, i, str(v), ha="center", va="center",
305               color="white" if v > cm.max() / 2 else "black")
306     ax.set_xticks(range(len(classes)), classes, rotation=45)
307     ax.set_yticks(range(len(classes)), classes)
308     ax.set_xlabel("prédit")
309     ax.set_ylabel("réel")
310     ax.set_title("Matrice de confusion")
311     col_g, col_d = st.columns(2)
312     col_g.pyplot(fig)

```

```

313 # --- visualisation spécifique ---
314 if algo == "Arbre de décision":
315     fig2, ax2 = plt.subplots(figsize=(9, 5))
316     plot_tree(model, feature_names=list(X.columns), class_names=model.classes_,
317               filled=True, rounded=True, fontsize=7, max_depth=3)
318     ax2.set_title("Arbre (3 premiers niveaux)")
319     col_d.pyplot(fig2)
320 elif algo == "Forêt aléatoire":
321     imp = pd.Series(model.feature_importances_, index=X.columns).sort_values()
322     col_d.bar_chart(imp)
323
324 st.success("Modèle entraîné. Le jeu a été coupé en apprentissage / test "
325           f"({100*(1-test_size):.0f}% / {100*test_size:.0f}%).",
326           icon=":material/check_circle:")
327 st.info("***Comparaison WEKA** : *Classify → trees/J48* (arbre), *lazy/IBk* "
328         "(K-NN) ou *trees/RandomForest*. Comparer l'*accuracy** et la matrice de "
329         "confusion de la sortie WEKA à celles ci-dessus.", icon=":material/science:")
330
331 # =====
332 # 3) RECHERCHE D'ASSOCIATIONS
333 # =====
334 else:
335     st.header(":material/shopping_cart: Recherche d'associations (Apriori)")
336     df = charger(demo_association)
337     st.dataframe(df.head(), width="stretch")
338
339     mode = st.radio(
340         "Format des données",
341         ["Colonne 'panier' (articles séparés par des virgules)",
342          "Tableau (chaque colonne catégorielle = un item 'colonne=valeur')"],
343     )
344     col_panier = None
345     if mode.startswith("Colonne"):
346         default = "panier" if "panier" in df.columns else df.columns[0]
347         col_panier = st.selectbox("Colonne contenant les paniers", df.columns,
348                                  index=list(df.columns).index(default))
349     c1, c2 = st.columns(2)
350     min_sup = c1.slider("Support minimum", 0.05, 1.0, 0.3, step=0.05)
351     min_conf = c2.slider("Confiance minimum", 0.1, 1.0, 0.6, step=0.05)
352
353     if st.button("Extraire les règles", type="primary",
354                 icon=":material/play_arrow:", width="stretch"):
355         # construire la liste de transactions
356         if mode.startswith("Colonne"):
357             transactions = [set(str(v).split(",")) for v in df[col_panier].dropna()]
358         else:
359             cat = df.select_dtypes(exclude="number").columns
360             transactions = [
361                 {f"{c}={row[c]}" for c in cat} for _, row in df.iterrows()
362             ]
363
364     regles = regles_association(transactions, min_sup, min_conf)
365     st.metric("Nombre de règles trouvées", len(regles))
366
367     if regles:
368         tab = pd.DataFrame(
369             [
370                 {
371                     "Si (antécédent)": ", ".join(sorted(r["antecedent"])),
372                     "Alors (conséquent)": ", ".join(sorted(r["consequent"])),
373                     "Support": r["support"],
374                     "Confiance": r["confidence"],
375                 }
376             ]

```

```

377         "Lift": r["lift"],
378     }
379     for r in regles
380 ]
381 )
382 st.dataframe(tab, width="stretch")
383 telecharger(tab, "regles_association.csv")
384
385 st.subheader("Top règles par confiance")
386 top = tab.head(10).copy()
387 top["règle"] = top["Si (antécédent)"] + " → " + top["Alors (conséquent)"]
388 st.bar_chart(top.set_index("règle")["Confiance"])
389 else:
390     st.warning("Aucune règle : baisse le support ou la confiance minimum.",
391               icon="material/warning:")
392
393 st.info("***Comparaison WEKA** : *Associate → Apriori*. Régler "
394         "`lowerBoundMinSupport` et `minMetric` (confiance) aux mêmes valeurs ; "
395         "comparer les règles et leurs *lift*.", icon="material/science:")
396
397 with st.expander("Rappels – support, confiance, lift", icon="material/menu_book:"):
398     st.latex(r"\text{support}(X)=\frac{\#\{\text{transactions} \supseteq X\}}{N}")
399     st.latex(r"\text{confiance}(X \text{ to } Y)=\frac{\text{support}(X \cup Y)}{\text{support}(X)}")
400     st.latex(r"\text{lift}(X \text{ to } Y)=\frac{\text{confiance}(X \text{ to } Y)}{\text{support}(Y)}")
401     r"\quad(>1:\text{corrélation positive})")

```

En résumé

L'application Streamlit couvre les **8 fonctionnalités** du sujet pour les **3 tâches**. Chaque tâche propose plusieurs algorithmes, affiche les **mesures de performance** adaptées et indique l'équivalent **WEKA** pour la comparaison. Apriori est implémenté « à la main » pour bien montrer le calcul du support, de la confiance et du lift.